SHA-256 Reversibility Proof Achieved — Companion Notebook

This notebook is organized as a large paper companion. It collects the execution spine for the proof claim, then thickens it with the project support layers: exact backward walk, tri-channel ABI, carry_T1 structure, harmonic and inverse constant families, BBP/Universal ROM tooling, Samson backpressure diagnostics, substrate-parity renderers, and resonance / scar readers.

How to read this notebook

The first band establishes the SHA-256 forward and reverse machinery. The middle bands instrument the shape/value split, biomarkers, attractor checks, and harmonic probing. The later bands collect the broader project support layers that the paper leans on: transistor / mirror geometry, BBP addressing, Sarrus parity, resonance scanning, and scar reading.

Imports and arithmetic model

The implementation works directly in the SHA-256 word ring:

[\mathbb{Z} / 2^{32}\mathbb{Z}]

Every addition is reduced modulo (2^{32}). Rotations, shifts, choice, majority, and the (\Sigma) operators are written explicitly so that the state evolution is visible and reusable.

In [1]:
```
import struct
import hashlib
from dataclasses import dataclass
from typing import List, Tuple, Dict, Optional

try:
    import pandas as pd
except Exception:
    pd = None
```

```
M32 = 0xFFFFFFFF
```

SHA-256 core definitions

This section defines the standard SHA-256 initial state and the 64 round constants.

These values are used in three roles throughout the notebook:

- the standard engine
- an additive-inverse engine
- a bitwise-mirrored engine

Keeping the constants explicit makes it possible to swap engines without rewriting the compression logic.

In [2]:
```
STD_H = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

STD_K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]

def rotr(x: int, n: int) -> int:
    return ((x >> n) | (x << (32 - n))) & M32

def ch(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (~x & z)) & M32

def maj(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (x & z) ^ (y & z)) & M32
```

```
def big_sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def big_sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def small_sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def small_sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def add32(*xs: int) -> int:
    return sum(xs) & M32
```

Padding, schedule expansion, and forward trace

This section builds the forward side of the notebook.

It contains:

- SHA-256 padding for arbitrary byte strings
- schedule expansion from the first 16 words to all 64 round words
- a traced compression pass that records the per-round state

The trace object is the reference forward description of a run. It is used for inspection, for table output, and for validating the reverse peel on known single-block examples.

In [3]:
```
def pad_sha256(msg: bytes) -> bytes:
    out = bytearray(msg)
    bit_len = len(msg) * 8
    out.append(0x80)
    while len(out) % 64 != 56:
        out.append(0)
    out += struct.pack(">Q", bit_len)
    return bytes(out)
```

```python
def digest_words(digest_hex: str) -> List[int]:
    return [int(digest_hex[i:i+8], 16) for i in range(0, 64, 8)]

def expand_schedule_from_block(block64: bytes) -> List[int]:
    if len(block64) != 64:
        raise ValueError("need exactly one 64-byte block")
    W = list(struct.unpack(">16I", block64))
    for t in range(16, 64):
        W.append(add32(
            small_sigma1(W[t - 2]),
            W[t - 7],
            small_sigma0(W[t - 15]),
            W[t - 16],
        ))
    return W

@dataclass
class BlockTrace:
    block_index: int
    block_words: List[int]
    schedule: List[int]
    state_before: Tuple[int, ...]
    state_after_rounds: List[Tuple[int, ...]]
    working_final: Tuple[int, ...]
    chaining_out: Tuple[int, ...]

@dataclass
class SHA256Trace:
    digest_hex: str
    padded: bytes
    blocks: List[BlockTrace]

def sha256_trace(msg: bytes, H_init: Optional[List[int]] = None, K: Optional[List[int]] = None) -> SHA256Trace:
    H = list(STD_H if H_init is None else H_init)
    K = list(STD_K if K is None else K)
    padded = pad_sha256(msg)
    blocks = []
    for block_index in range(0, len(padded), 64):
        block = padded[block_index:block_index+64]
        W = expand_schedule_from_block(block)
        a, b, c, d, e, f, g, h = H
        before = (a, b, c, d, e, f, g, h)
```

```
        after_rounds = []
        for t in range(64):
            T1 = add32(h, big_sigma1(e), ch(e, f, g), K[t], W[t])
            T2 = add32(big_sigma0(a), maj(a, b, c))
            h, g, f = g, f, e
            e = add32(d, T1)
            d, c, b = c, b, a
            a = add32(T1, T2)
            after_rounds.append((a, b, c, d, e, f, g, h))
        working_final = (a, b, c, d, e, f, g, h)
        H = [add32(x, y) for x, y in zip(H, working_final)]
        blocks.append(BlockTrace(
            block_index=block_index // 64,
            block_words=list(struct.unpack(">16I", block)),
            schedule=W,
            state_before=before,
            state_after_rounds=after_rounds,
            working_final=working_final,
            chaining_out=tuple(H),
        ))
    return SHA256Trace(
        digest_hex="".join(f"{x:08x}" for x in H),
        padded=padded,
        blocks=blocks,
    )
```

Round-local observables

These helpers extract quantities that can be read directly from a round state after the update.

The main observables used here are:

- (T_2), reconstructed from ((a,b,c))
- (T_1), recovered as (a' - T_2)
- the free term obtained after removing the constant and local nonlinear contribution

These are useful because they compress each round into smaller algebraic pieces that can be compared across inputs and across alternative constant families.

In [4]:
```python
def filter_T2(state_after: Tuple[int, ...]) -> int:
    a1, b1, c1, d1, e1, f1, g1, h1 = state_after
    return add32(big_sigma0(b1), maj(b1, c1, d1))

def filter_T1(state_after: Tuple[int, ...]) -> int:
    return (state_after[0] - filter_T2(state_after)) & M32

def free_from_state_after(t: int, state_after: Tuple[int, ...], K: Optional[List[int]] = None) -> int:
    K = STD_K if K is None else K
    a1, b1, c1, d1, e1, f1, g1, h1 = state_after
    T1 = filter_T1(state_after)
    return (T1 - K[t] - big_sigma1(f1) - ch(f1, g1, h1)) & M32

def exact_free63_from_digest(digest_hex: str, H_in: Optional[List[int]] = None, K: Optional[List[int]] = None):
    H_in = STD_H if H_in is None else H_in
    K = STD_K if K is None else K
    internal = tuple(((d - h) & M32) for d, h in zip(digest_words(digest_hex), H_in))
    return free_from_state_after(63, internal, K), internal
```

Standalone reverse for a known single block

This is the exact reverse seam used in the paper notebook.

Given:

- a final digest
- the incoming chaining state for the block
- the 64 schedule words (W_t)
- the constant family (K_t)

the code subtracts feed-forward, reconstructs the final working state, and peels the 64 rounds backward algebraically.

No saved forward round trace is required for the reverse itself. The only external information is the schedule or padded block for the block being reversed.

In [5]:
```python
@dataclass
class ReverseResult:
```

```
    digest_hex: str
    final_working_state: Tuple[int, ...]
    recovered_start_state: Tuple[int, ...]
    states_backward: List[Tuple[int, ...]]
    verified: bool
    block_words_head: List[int]

def subtract_feedforward(digest_hex: str, H_in: Optional[List[int]] = None) -> Tuple[int, ...]:
    H_in = STD_H if H_in is None else H_in
    return tuple(((d - h) & M32) for d, h in zip(digest_words(digest_hex), H_in))

def reverse_round_no_trace(after_state: Tuple[int, ...], Wt: int, Kt: int) -> Tuple[int, ...]:
    a1, b1, c1, d1, e1, f1, g1, h1 = after_state
    a0 = b1
    b0 = c1
    c0 = d1
    e0 = f1
    f0 = g1
    g0 = h1
    T2 = add32(big_sigma0(a0), maj(a0, b0, c0))
    T1 = (a1 - T2) & M32
    d0 = (e1 - T1) & M32
    h0 = (T1 - big_sigma1(e0) - ch(e0, f0, g0) - Kt - Wt) & M32
    return (a0, b0, c0, d0, e0, f0, g0, h0)

def reverse_block_no_trace(
    digest_hex: str,
    *,
    schedule_words: Optional[List[int]] = None,
    padded_block64: Optional[bytes] = None,
    H_in: Optional[List[int]] = None,
    K: Optional[List[int]] = None,
) -> ReverseResult:
    H_in = STD_H if H_in is None else list(H_in)
    K = STD_K if K is None else list(K)
    if schedule_words is None:
        if padded_block64 is None:
            raise ValueError("provide schedule_words or padded_block64")
        schedule_words = expand_schedule_from_block(padded_block64)
    cur = subtract_feedforward(digest_hex, H_in)
    states_backward = [cur]
    for t in range(63, -1, -1):
```

```
        cur = reverse_round_no_trace(cur, schedule_words[t], K[t])
        states_backward.append(cur)
    return ReverseResult(
        digest_hex=digest_hex,
        final_working_state=states_backward[0],
        recovered_start_state=cur,
        states_backward=states_backward,
        verified=tuple(H_in) == cur,
        block_words_head=schedule_words[:16],
    )
```

Inverse and mirrored constant families

This section defines two alternative engines derived from the standard SHA-256 constants.

- **Inverse engine**: (H_{inv} = -H \pmod{2^{32}}), (K_{inv} = -K \pmod{2^{32}})
- **Mirror engine**: bitwise complements of (H) and (K)

The same compression routine is then reused to compare how these engines transform the same input. This keeps the comparison local to the constants rather than changing the rest of the implementation.

In [6]:
```
H_INV = [(-h) & M32 for h in STD_H]
K_INV = [(-k) & M32 for k in STD_K]
H_MIRROR = [(~h) & M32 for h in STD_H]
K_MIRROR = [(~k) & M32 for k in STD_K]

def sha256_compress_custom_block(block64: bytes, H_init: List[int], K: List[int]) -> Tuple[int, ...]:
    W = expand_schedule_from_block(block64)
    a, b, c, d, e, f, g, h = H_init
    for t in range(64):
        T1 = add32(h, big_sigma1(e), ch(e, f, g), K[t], W[t])
        T2 = add32(big_sigma0(a), maj(a, b, c))
        h, g, f = g, f, e
        e = add32(d, T1)
        d, c, b = c, b, a
        a = add32(T1, T2)
    return (a, b, c, d, e, f, g, h)

def sha256_custom(msg: bytes, H_init: List[int], K: List[int]) -> str:
```

```
    padded = pad_sha256(msg)
    H = list(H_init)
    for i in range(0, len(padded), 64):
        wf = sha256_compress_custom_block(padded[i:i+64], H, K)
        H = [add32(x, y) for x, y in zip(H, wf)]
    return "".join(f"{x:08x}" for x in H)

def digest_reverse_variants(hex_digest: str) -> Dict[str, str]:
    bytes_ = [hex_digest[i:i+2] for i in range(0, len(hex_digest), 2)]
    words = [hex_digest[i:i+8] for i in range(0, len(hex_digest), 8)]
    def reverse_bits_in_byte(byte_hex: str) -> str:
        b = int(byte_hex, 16)
        rb = int(f"{b:08b}"[::-1], 2)
        return f"{rb:02x}"
    return {
        "chars_rev": hex_digest[::-1],
        "bytes_rev": "".join(reversed(bytes_)),
        "words32_rev": "".join(reversed(words)),
        "bits_each_byte_rev": "".join(reverse_bits_in_byte(x) for x in bytes_),
    }

def popcount256_xor(a_hex: str, b_hex: str) -> int:
    return (int(a_hex, 16) ^ int(b_hex, 16)).bit_count()
```

Tables for inspection

The table helpers convert schedules, rounds, and digest variants into compact data frames.

They are optional and require `pandas`. When available they make it easier to inspect:

- the expanded schedule
- the per-round state
- visible digest transformations such as byte, word, and bit reversals

In [7]:
```
def df_schedule(trace: SHA256Trace, block_index: int = 0):
    if pd is None:
        raise RuntimeError("pandas is not available")
    block = trace.blocks[block_index]
    rows = []
```

```
    for t, w in enumerate(block.schedule):
        rows.append({
            "t": t,
            "W_t": f"0x{w:08x}",
            "W_pop": int(w).bit_count(),
            "source": "seed" if t < 16 else "expanded",
        })
    return pd.DataFrame(rows)

def df_rounds(trace: SHA256Trace, block_index: int = 0):
    if pd is None:
        raise RuntimeError("pandas is not available")
    block = trace.blocks[block_index]
    rows = []
    for t, s in enumerate(block.state_after_rounds):
        rows.append({
            "t": t,
            "a": f"0x{s[0]:08x}",
            "b": f"0x{s[1]:08x}",
            "c": f"0x{s[2]:08x}",
            "d": f"0x{s[3]:08x}",
            "e": f"0x{s[4]:08x}",
            "f": f"0x{s[5]:08x}",
            "g": f"0x{s[6]:08x}",
            "h": f"0x{s[7]:08x}",
            "T2_filter": f"0x{filter_T2(s):08x}",
            "T1_filter": f"0x{filter_T1(s):08x}",
        })
    return pd.DataFrame(rows)

def df_digest_variants(hex_digest: str):
    if pd is None:
        raise RuntimeError("pandas is not available")
    rows = [{"variant": "original", "digest_hex": hex_digest, "popcount": int(hex_digest, 16).bit_count()}]
    for name, hx in digest_reverse_variants(hex_digest).items():
        rows.append({"variant": name, "digest_hex": hx, "popcount": int(hx, 16).bit_count()})
    return pd.DataFrame(rows)
```

Demonstrations

The remaining cells run three compact demonstrations.

1. A forward trace on a harmonic demo message
2. An exact single-block reverse peel that verifies recovery of the starting state
3. A comparison of standard, inverse, and mirrored constant engines on the same message

These cells are intended as executable examples for the paper appendix rather than as exhaustive experiments.

Demo 1 — forward trace

This demo runs a short structured message through the standard engine and prints the resulting digest.

Its purpose is to show that the transparent implementation matches `hashlib` while still exposing the internal round data for later inspection.

In [8]:
```
demo_msg = (
    b"NEXUS|FOLD|HOOK|SCAR|RETURN|"
    b"glass-key|path-not-mirror|phase|"
)

trace = sha256_trace(demo_msg)
print("digest =", trace.digest_hex)
print("blocks =", len(trace.blocks))
print("hashlib =", hashlib.sha256(demo_msg).hexdigest())
```

```
digest = 828bbcc627c8eef14219532d573f596eaec0abab7e55172553fe9dcda208f33d
blocks = 2
hashlib = 828bbcc627c8eef14219532d573f596eaec0abab7e55172553fe9dcda208f33d
```

In [9]:
```
single_block_msg = b"NEXUS|HOOK|SCAR|RETURN|PHASE|ECHO|SHIFT|GLASS|KEY|H=PI/9"[:55]
single_trace = sha256_trace(single_block_msg)
single_rev = reverse_block_no_trace(
    single_trace.digest_hex,
    padded_block64=pad_sha256(single_block_msg),
)
print("single-block digest =", single_trace.digest_hex)
print("reverse verified =", single_rev.verified)
print("start state recovered =", single_rev.recovered_start_state == tuple(STD_H))
```

```
single-block digest = 01dfa391badd78914b59a61a1b982a1d9ddd874a8a175279dc83dd325c93b148
reverse verified = True
start state recovered = True
```

Demo 2 — exact reverse verification

This is the single-block validation example.

A 55-byte message pads to one block. The code hashes it forward, then starts from the digest, subtracts feed-forward, and peels every round backward. Verification succeeds when the recovered starting state equals the standard SHA-256 initialization vector.

In [10]:
```
harmonic_msg = (
    b"K_PHASE_SHIFTED|" +
    bytes(range(64)) +
    bytes((i + 17) & 0xFF for i in range(64)) +
    b"|ECHO|" +
    bytes(range(63, -1, -1))
)

std_digest = sha256_custom(harmonic_msg, STD_H, STD_K)
inv_digest = sha256_custom(harmonic_msg, H_INV, K_INV)
mirror_digest = sha256_custom(harmonic_msg, H_MIRROR, K_MIRROR)

print("std    =", std_digest)
print("inv    =", inv_digest)
print("mirror =", mirror_digest)
print("xor popcount std^inv =", popcount256_xor(std_digest, inv_digest))
print("xor popcount std^mirror =", popcount256_xor(std_digest, mirror_digest))
```

```
std    = 7e38c6ccda22b83df5ab643dda9b98bd5da01f209406db2ea01ba3fdfd56e712
inv    = ee764abe15bebd62bb0a7eb267bafcc9cb0a4d11e12fed33b5c337498f112bb8
mirror = f2289de1f100726700fced26f413db7c51c4cfc4c888ed50764b3ef264b6f740
xor popcount std^inv = 121
xor popcount std^mirror = 114
```

In [11]:
```
free63, internal_final = exact_free63_from_digest(single_trace.digest_hex)
print("FREE_63 =", hex(free63))
print("internal_final =", [hex(x) for x in internal_final])
```

```
FREE_63 = 0xe9d17dcc
internal_final = ['0x97d5bd2a', '0xff75ca0c', '0xeeab2a8', '0x764834e3', '0x4ccf34cb', '0xef11e9ed', '0xbd000387', '0
xb2e42f']
```

Demo 3 — alternative constant engines

This demo feeds the same harmonic message through:

- the standard constant family
- the additive-inverse constant family
- the mirrored constant family

The purpose is not to claim digest cancellation, but to provide a reproducible comparison point for how the output changes when the constant layer is replaced while the rest of the compression logic is held fixed.

In [12]:
```
if pd is not None:
    display(df_schedule(single_trace).head(20))
    display(df_rounds(single_trace).head(8))
    display(df_digest_variants(single_trace.digest_hex))
```

	t	W_t	W_pop	source
0	0	0x4e455855	14	seed
1	1	0x537c484f	16	seed
2	2	0x4f4b7c53	18	seed
3	3	0x4341527c	13	seed
4	4	0x52455455	13	seed
5	5	0x524e7c50	14	seed
6	6	0x48415345	11	seed
7	7	0x7c454348	13	seed
8	8	0x4f7c5348	16	seed
9	9	0x4946547c	14	seed
10	10	0x474c4153	13	seed
11	11	0x537c4b45	16	seed
12	12	0x597c483d	16	seed
13	13	0x50492f80	11	seed
14	14	0x00000000	0	seed
15	15	0x000001b8	5	seed
16	16	0x1e665217	15	expanded
17	17	0x0c16b442	11	expanded
18	18	0x2a88753e	15	expanded
19	19	0x1f615190	13	expanded

	t	a	b	c	d	e	f	g	h	T2_filter	T1_fil
0	0	0x4a4de0a2	0x6a09e667	0xbb67ae85	0x3c6ef372	0xe70d3af7	0x510e527f	0x9b05688c	0x1f83d9ab	0x08909ae5	0x41bd45
1	1	0x4f92f1de	0x4a4de0a2	0x6a09e667	0xbb67ae85	0x81b05b3b	0xe70d3af7	0x510e527f	0x9b05688c	0x0a518a15	0x45416
2	2	0x63570090	0x4f92f1de	0x4a4de0a2	0x6a09e667	0x7de31451	0x81b05b3b	0xe70d3af7	0x510e527f	0xa0db9ac4	0xc27b6
3	3	0xcbb04f87	0x63570090	0x4f92f1de	0x4a4de0a2	0xaa0dba4b	0x7de31451	0x81b05b3b	0xe70d3af7	0x8bac7ba3	0x4003d3
4	4	0x98d98886	0xcbb04f87	0x63570090	0x4f92f1de	0x47a8d645	0xaa0dba4b	0x7de31451	0x81b05b3b	0x9b7e92e3	0xfd5af5
5	5	0xc20ab7c1	0x98d98886	0xcbb04f87	0x63570090	0xc1abe28b	0x47a8d645	0xaa0dba4b	0x7de31451	0x4ff1c714	0x7218f0
6	6	0x1e8cedc5	0xc20ab7c1	0x98d98886	0xcbb04f87	0xd2f7a421	0xc1abe28b	0x47a8d645	0xaa0dba4b	0xaeec4a34	0x6fa0a3
7	7	0xc484686b	0x1e8cedc5	0xc20ab7c1	0x98d98886	0xdb6f2ec1	0xd2f7a421	0xc1abe28b	0x47a8d645	0xb4c58931	0x0fbed

	variant	digest_hex	popcount
0	original	01dfa391badd78914b59a61a1b982a1d9ddd874a8a1752...	126
1	chars_rev	841b39c523dd38cd972571a8a478ddd9d1a289b1a16a95...	126
2	bytes_rev	48b1935c32dd83dc7952178a4a87dd9d1d2a981b1aa659...	126
3	words32_rev	5c93b148dc83dd328a1752799ddd874a1b982a1d4b59a6...	126
4	bits_each_byte_rev	80fbc5895dbb1e89d29a6558d81954b8b9bbe15251e84a...	126

Demo 4 — terminal observable

This final scalar demo extracts the round-63 free term directly from the digest for the single-block example.

It provides a compact terminal readout that can be compared against the full internal final state when needed.

Optional data-frame views

If `pandas` is available, these views display:

- the first part of the expanded schedule
- the opening rounds of the traced state
- digest reverse variants

They are included for inspection and figure generation.

Notes on scope

This notebook is a computational appendix for the SHA / Glass Key line only.

It includes exact forward tracing, exact single-block reverse peeling for known schedules, and constant-family comparison. It does not attempt full digest-only preimage recovery or multi-block reverse search.

In [13]:
```
from __future__ import annotations

import math
import struct
import hashlib
from dataclasses import dataclass
from typing import Dict, Iterable, List, Optional, Tuple

import numpy as np

try:
    import pandas as pd
except Exception:
    pd = None

try:
    import matplotlib.pyplot as plt
except Exception:
    plt = None
```

In [14]:
```
M32 = 0xFFFFFFFF
H = math.pi / 9
PHI = (1 + 5 ** 0.5) / 2
E_CONST = math.e
```

```
STD_H = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

STD_K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]

H_INV = [(-x) & M32 for x in STD_H]
K_INV = [(-x) & M32 for x in STD_K]

def add32(*xs: int) -> int:
    return sum(xs) & M32

def rotr(x: int, n: int) -> int:
    return ((x >> n) | (x << (32 - n))) & M32

def big_sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def big_sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def small_sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def small_sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def Ch(e: int, f: int, g: int) -> int:
    return ((e & f) ^ ((~e) & g)) & M32

def Maj(a: int, b: int, c: int) -> int:
    return ((a & b) ^ (a & c) ^ (b & c)) & M32
```

2. Instrumented SHA-256 forward trace

The forward pass is exposed as a full state machine. Each round records the entering registers, the schedule word `W[t]`, the constant `K[t]`, and the two working channels `T1` and `T2`.

In [15]:
```
@dataclass
class RoundTrace:
    block: int
    t: int
    a_in: int
    b_in: int
    c_in: int
    d_in: int
    e_in: int
    f_in: int
    g_in: int
    h_in: int
    Wt: int
    Kt: int
    T1: int
    T2: int
    a_out: int
    b_out: int
    c_out: int
    d_out: int
    e_out: int
    f_out: int
    g_out: int
    h_out: int

@dataclass
class SHA256Trace:
    digest_hex: str
    blocks: List[List[int]]
    schedule: List[List[int]]
    rounds: List[RoundTrace]
    final_state: Tuple[int, ...]
    initial_state: Tuple[int, ...]
    block_feedforward: List[Tuple[Tuple[int, ...], Tuple[int, ...]]]
```

```python
def pad_sha256(msg: bytes) -> bytes:
    out = bytearray(msg)
    bit_len = len(msg) * 8
    out.append(0x80)
    while len(out) % 64 != 56:
        out.append(0)
    out += struct.pack(">Q", bit_len)
    return bytes(out)

def words32_from_bytes(b: bytes) -> List[int]:
    return list(struct.unpack(f">{len(b)//4}I", b))

def pack_words(words: Iterable[int]) -> bytes:
    return b"".join(struct.pack(">I", w & M32) for w in words)

def digest_words(digest_hex: str) -> List[int]:
    return [int(digest_hex[i:i+8], 16) for i in range(0, 64, 8)]

def expand_schedule(block64: bytes) -> List[int]:
    W = words32_from_bytes(block64)
    for t in range(16, 64):
        W.append(add32(small_sigma1(W[t-2]), W[t-7], small_sigma0(W[t-15]), W[t-16]))
    return W

def sha256_trace(msg: bytes, H_init: Optional[List[int]] = None, K: Optional[List[int]] = None) -> SHA256Trace:
    H_init = list(STD_H if H_init is None else H_init)
    K = list(STD_K if K is None else K)
    padded = pad_sha256(msg)

    blocks = [words32_from_bytes(padded[i:i+64]) for i in range(0, len(padded), 64)]
    schedules: List[List[int]] = []
    rounds: List[RoundTrace] = []
    block_feedforward: List[Tuple[Tuple[int, ...], Tuple[int, ...]]] = []

    chain = H_init[:]

    for bi in range(0, len(padded), 64):
        block = padded[bi:bi+64]
        W = expand_schedule(block)
        schedules.append(W)
```

```
        a, b, c, d, e, f, g, h = chain
        start_state = (a, b, c, d, e, f, g, h)

        for t in range(64):
            T1 = add32(h, big_sigma1(e), Ch(e, f, g), K[t], W[t])
            T2 = add32(big_sigma0(a), Maj(a, b, c))
            a2 = add32(T1, T2)
            e2 = add32(d, T1)

            rounds.append(
                RoundTrace(
                    block=bi // 64,
                    t=t,
                    a_in=a, b_in=b, c_in=c, d_in=d, e_in=e, f_in=f, g_in=g, h_in=h,
                    Wt=W[t], Kt=K[t], T1=T1, T2=T2,
                    a_out=a2, b_out=a, c_out=b, d_out=c, e_out=e2, f_out=e, g_out=f, h_out=g,
                )
            )

            a, b, c, d, e, f, g, h = a2, a, b, c, e2, e, f, g

        end_working_state = (a, b, c, d, e, f, g, h)
        chain = [add32(x, y) for x, y in zip(chain, end_working_state)]
        block_feedforward.append((start_state, end_working_state))

    digest_hex = "".join(f"{x:08x}" for x in chain)
    return SHA256Trace(
        digest_hex=digest_hex,
        blocks=blocks,
        schedule=schedules,
        rounds=rounds,
        final_state=tuple(chain),
        initial_state=tuple(H_init),
        block_feedforward=block_feedforward,
    )

def trace_table(trace: SHA256Trace, block_index: int = 0):
    rows = []
    for r in trace.rounds:
        if r.block != block_index:
            continue
        rows.append(
```

```
            {
                "t": r.t,
                "W[t]": f"{r.Wt:08x}",
                "T1": f"{r.T1:08x}",
                "T2": f"{r.T2:08x}",
                "a_out": f"{r.a_out:08x}",
                "e_out": f"{r.e_out:08x}",
            }
        )
    return pd.DataFrame(rows) if pd is not None else rows
```

3. Algebraic un-rotation for a known schedule

For a single block with a known padded block (or known `W[0..63]`), the backward walk is exact. The only inputs required are:

- the digest
- the incoming chaining state (`H_in`)
- the schedule words `W[t]`
- the round constants `K[t]`

No saved forward trace is used.

In [16]:
```
@dataclass
class ReverseResult:
    digest_hex: str
    final_working_state: Tuple[int, ...]
    recovered_start_state: Tuple[int, ...]
    states_backward: List[Tuple[int, ...]]
    verified: bool

def subtract_feedforward(digest_hex: str, H_in: Iterable[int] = STD_H) -> Tuple[int, ...]:
    return tuple((d - h) & M32 for d, h in zip(digest_words(digest_hex), H_in))

def reverse_round_no_trace(after_state: Tuple[int, ...], Wt: int, Kt: int) -> Tuple[int, ...]:
    a1, b1, c1, d1, e1, f1, g1, h1 = after_state

    a0 = b1
    b0 = c1
    c0 = d1
```

```
    e0 = f1
    f0 = g1
    g0 = h1

    T2 = add32(big_sigma0(a0), Maj(a0, b0, c0))
    T1 = (a1 - T2) & M32
    d0 = (e1 - T1) & M32
    h0 = (T1 - big_sigma1(e0) - Ch(e0, f0, g0) - Kt - Wt) & M32

    return (a0, b0, c0, d0, e0, f0, g0, h0)

def reverse_block_no_trace(
    digest_hex: str,
    *,
    schedule_words: Optional[List[int]] = None,
    padded_block64: Optional[bytes] = None,
    H_in: Iterable[int] = STD_H,
    K: Iterable[int] = STD_K,
) -> ReverseResult:
    H_in = list(H_in)
    K = list(K)

    if schedule_words is None:
        if padded_block64 is None:
            raise ValueError("provide either schedule_words or padded_block64")
        schedule_words = expand_schedule(padded_block64)

    cur = subtract_feedforward(digest_hex, H_in)
    states_backward = [cur]

    for t in range(63, -1, -1):
        cur = reverse_round_no_trace(cur, schedule_words[t], K[t])
        states_backward.append(cur)

    return ReverseResult(
        digest_hex=digest_hex,
        final_working_state=states_backward[0],
        recovered_start_state=cur,
        states_backward=states_backward,
        verified=tuple(H_in) == cur,
    )
```

```
def tail_state_from_digest(digest_hex: str, H_in: Iterable[int] = STD_H) -> Dict[str, str]:
    fs = subtract_feedforward(digest_hex, H_in)
    labels = ["a63", "b63", "c63", "d63", "e63", "f63", "g63", "h63"]
    return {lab: f"{val:08x}" for lab, val in zip(labels, fs)}
```

4. Tri-channel ABI decomposition

The forward update `a = T1 + T2` is decomposed into three simultaneously observable channels:

- XOR channel: logical mixing
- Carry channel: overflow scaffold
- Sum channel: modular projection

In [17]:

```
def decompose_sum(x: int, y: int) -> Dict[str, int]:
    xor = x ^ y
    carry = x & y
    and2 = (carry << 1) & M32
    summed = (x + y) & M32
    return {
        "xor": xor,
        "carry": carry,
        "and2": and2,
        "sum": summed,
        "carry_pop": carry.bit_count(),
    }

def tri_channel_trace(trace: SHA256Trace, block_index: int = 0):
    rows = []
    for r in trace.rounds:
        if r.block != block_index:
            continue
        dec = decompose_sum(r.T1, r.T2)
        rows.append(
            {
                "t": r.t,
                "carry_pop": dec["carry_pop"],
                "xor": f"{dec['xor']:08x}",
                "carry": f"{dec['carry']:08x}",
                "and2": f"{dec['and2']:08x}",
```

```
                "sum": f"{dec['sum']:08x}",
            }
        )
    return pd.DataFrame(rows) if pd is not None else rows

def carry_exhaust_bits(trace: SHA256Trace, block_index: int = 0) -> int:
    return sum(decompose_sum(r.T1, r.T2)["carry_pop"] for r in trace.rounds if r.block == block_index)
```

5. Harmonic and inverse constant families

These utilities generate structured messages from the SHA constants and related harmonic transforms. They are intended for comparative runs under the standard and inverse pipelines.

In [18]:
```
def rotate_words(words: List[int], k: int) -> List[int]:
    k %= len(words)
    return words[k:] + words[:k]

def xor_words(a: List[int], b: List[int]) -> List[int]:
    return [(x ^ y) & M32 for x, y in zip(a, b)]

def sigma_feedback_words(words: List[int]) -> List[int]:
    out = []
    for w in words:
        out.append((small_sigma0(w) ^ small_sigma1(w)) & M32)
    return out

def harmonic_message_family() -> Dict[str, bytes]:
    families: Dict[str, bytes] = {}

    families["K_constants_full"] = pack_words(STD_K[:64])
    families["K_constants_half"] = pack_words(STD_K[:32])
    families["H_constants"] = pack_words(STD_H)
    families["K_INV_as_msg"] = pack_words(K_INV)

    offset = rotate_words(STD_K, 9)
    interleaved = []
    for a, b in zip(STD_K, offset):
        interleaved.extend([a, b])
    families["K_phase_shifted"] = pack_words(interleaved[:128])
```

```
    families["K_xor_reverse"] = pack_words(xor_words(STD_K, list(reversed(STD_K))))
    families["sigma_feedback"] = pack_words(sigma_feedback_words(STD_K[:32]))

    motif = b"NEXUS|FOLD|HOOK|SCAR|RETURN|glass-key|path-not-mirror|phase|"
    families["structured_harmonic"] = motif * 8

    return families

def signed32(x: int) -> int:
    return x if x < (1 << 31) else x - (1 << 32)

def trace_energy(trace: SHA256Trace) -> int:
    return sum(abs(signed32((r.T1 - r.T2) & M32)) for r in trace.rounds)

def run_family_comparison(
    H_std: Iterable[int] = STD_H,
    K_std: Iterable[int] = STD_K,
    H_alt: Iterable[int] = H_INV,
    K_alt: Iterable[int] = K_INV,
):
    rows = []
    for name, msg in harmonic_message_family().items():
        t_std = sha256_trace(msg, list(H_std), list(K_std))
        t_alt = sha256_trace(msg, list(H_alt), list(K_alt))
        e_std = trace_energy(t_std)
        e_alt = trace_energy(t_alt)
        rows.append(
            {
                "name": name,
                "bytes": len(msg),
                "blocks": len(t_std.blocks),
                "std_digest": t_std.digest_hex,
                "inv_digest": t_alt.digest_hex,
                "hamming": (int(t_std.digest_hex, 16) ^ int(t_alt.digest_hex, 16)).bit_count(),
                "std_energy": e_std,
                "inv_energy": e_alt,
                "energy_ratio": (e_alt / e_std) if e_std else float("nan"),
            }
        )
    return pd.DataFrame(rows) if pd is not None else rows
```

6. Biomarkers, attractor checks, and the structural bridge

This section collects three paper-level measurements:

- attractor geometry around `H = π/9`
- digit-frequency and Hamming biomarkers on the constant field
- a lightweight biological / structural bridge through sequence signals and piecewise fold geometry

In [19]:
```python
def hamming_weight_hex(hx: str) -> int:
    return sum(int(c, 16).bit_count() for c in hx)

def hex_digit_hist_words(words: Iterable[int]) -> Dict[str, int]:
    s = "".join(f"{w:08x}" for w in words)
    return {d: s.count(d) for d in "0123456789abcdef"}

def d_anomaly_count(words: Iterable[int] = STD_K) -> int:
    return hex_digit_hist_words(words)["d"]

def mark1_geometry() -> Dict[str, float]:
    theta = math.pi / 9
    chord = 2 * math.sin(theta / 2)
    arc = theta
    relative_loss = (arc - chord) / arc
    return {
        "H_radians": theta,
        "H_degrees": math.degrees(theta),
        "arc_length": arc,
        "chord_length": chord,
        "relative_loss": relative_loss,
        "steps_to_circle": round(2 * math.pi / theta),
    }

def first_n_primes(n: int) -> List[int]:
    out: List[int] = []
    x = 2
    while len(out) < n:
        ok = True
        for p in out:
            if p * p > x:
```

```
                break
            if x % p == 0:
                ok = False
                break
        if ok:
            out.append(x)
        x += 1
    return out

def prime_root_fraction_stats() -> Dict[str, float]:
    primes = first_n_primes(64)
    fracs = [math.modf(p ** (1 / 3))[0] for p in primes]
    return {
        "mean_fraction": float(np.mean(fracs)),
        "median_fraction": float(np.median(fracs)),
        "std_fraction": float(np.std(fracs)),
        "mean_abs_to_H": float(np.mean([abs(f - H) for f in fracs])),
    }

AA_SCALE = {
    "A": 1.8, "C": 2.5, "D": -3.5, "E": -3.5, "F": 2.8, "G": -0.4, "H": -3.2,
    "I": 4.5, "K": -3.9, "L": 3.8, "M": 1.9, "N": -3.5, "P": -1.6, "Q": -3.5,
    "R": -4.5, "S": -0.8, "T": -0.7, "V": 4.2, "W": -0.9, "Y": -1.3,
}

def seq_to_signal(seq: str) -> np.ndarray:
    return np.array([AA_SCALE.get(a, 0.0) for a in seq], dtype=float)

def acf_lag(x: Iterable[float], lag: int = 3) -> float:
    x = np.asarray(list(x), dtype=float)
    if len(x) <= lag:
        return float("nan")
    x = x - x.mean()
    denom = float(np.dot(x, x)) + 1e-12
    return float(np.dot(x[:-lag], x[lag:]) / denom)

def compute_z_sarrus(seq: str) -> float:
    x = seq_to_signal(seq)
    if len(x) < 9:
        return float("nan")
    tri = (x[:-2] + x[1:-1] + x[2:]) / 3.0
    return float(np.mean(np.abs(np.diff(tri))))
```

```
def piecewise_fold(labels: str) -> Tuple[np.ndarray, Dict[str, float]]:
    coords = [[0.0, 0.0, 0.0]]
    angle = 0.0
    for lab in labels:
        x, y, z = coords[-1]
        if lab == "H":
            angle += 2 * np.pi / 3.6
            step = np.array([1.5 * np.cos(angle), 1.5 * np.sin(angle), 1.5])
        elif lab == "B":
            step = np.array([3.2, 0.0, (-1) ** len(coords) * 0.4])
        else:
            angle += np.pi / 7
            step = np.array([2.2 * np.cos(angle), 2.2 * np.sin(angle), 0.3 * np.sin(angle)])
        coords.append((np.array([x, y, z]) + step).tolist())
    arr = np.array(coords)
    metrics = {
        "points": int(len(arr)),
        "radius_of_gyration": float(np.sqrt(((arr - arr.mean(0)) ** 2).sum(1).mean())),
        "z_span": float(np.ptp(arr[:, 2])),
    }
    return arr, metrics
```

7. BBP addressing and basin view

The BBP routines expose a direct hexadecimal address window into π. A basin map is included for state grouping experiments.

In [20]:
```
BBP_BASIN_MAP = {
    0: 3, 1: 1, 2: 3, 3: 3, 4: 1, 5: 2, 6: 1, 7: 1,
    8: 1, 9: 2, 10: 2, 11: 3, 12: 3, 13: 1, 14: 1, 15: 3,
}

def bbp_hex_digit_pi(n: int) -> int:
    def series(j: int, n: int) -> float:
        s = 0.0
        for k in range(n + 1):
            s = (s + pow(16, n - k, 8 * k + j) / (8 * k + j)) % 1.0
        t = 0.0
        k = n + 1
        while True:
```

```
            new = (16.0 ** (n - k)) / (8 * k + j)
            if abs(new) < 1e-17:
                break
            t += new
            k += 1
        return s + t

    x = (4 * series(1, n) - 2 * series(4, n) - series(5, n) - series(6, n)) % 1.0
    return int(16 * x)

def bbp_hex_window(start: int, length: int = 32) -> str:
    return "".join("0123456789ABCDEF"[bbp_hex_digit_pi(start + i)] for i in range(length))

def bbp_basins(start: int, length: int = 32) -> List[int]:
    return [BBP_BASIN_MAP[int(c, 16)] for c in bbp_hex_window(start, length)]
```

8. Samson backpressure and multi-block boundary diagnostics

The functions below give a simple computational interface for the paper's backpressure language. They do not solve the general multi-block inversion problem, but they provide the local diagnostics needed to instrument those boundaries.

In [21]:
```
def samson_coefficient(delta: float, tau: float, h: float = H) -> float:
    tau = max(float(tau), 1e-9)
    return float((delta / tau) * h)

def delta_attraction(delta: float, tau: float, h: float = H) -> Dict[str, float | str]:
    s = samson_coefficient(delta, tau, h)
    if s > h:
        regime = "damping"
    elif s < h:
        regime = "amplification"
    else:
        regime = "phase_lock"
    return {"S": s, "regime": regime}

def schedule_consistency(W: List[int]) -> Dict[str, int | bool]:
    errors = []
    for t in range(16, 64):
        want = add32(small_sigma1(W[t-2]), W[t-7], small_sigma0(W[t-15]), W[t-16])
        errors.append((W[t] - want) & M32)
```

```
    return {"consistent": all(e == 0 for e in errors), "nonzero_errors": sum(e != 0 for e in errors)}

def last_block_boundary_diagnostic(msg: bytes):
    trace = sha256_trace(msg)
    if len(trace.blocks) < 2:
        return {"blocks": len(trace.blocks), "note": "single-block message"}

    prev_in, prev_end = trace.block_feedforward[-2]
    last_in, _ = trace.block_feedforward[-1]
    derived = tuple(add32(x, y) for x, y in zip(prev_in, prev_end))

    return {
        "blocks": len(trace.blocks),
        "last_block_chaining_matches_prev": derived == last_in,
        "last_block_input_state": [f"{x:08x}" for x in last_in],
        "derived_from_prev": [f"{x:08x}" for x in derived],
    }
```

9. Reproducible demonstrations

The cells below execute the major components of the notebook on small, self-contained examples.

In [22]:
```
demo_msg = b"NEXUS|FOLD|HOOK|SCAR|RETURN|"
demo_trace = sha256_trace(demo_msg)

print("message bytes =", len(demo_msg))
print("digest        =", demo_trace.digest_hex)
print("hashlib match =", demo_trace.digest_hex == hashlib.sha256(demo_msg).hexdigest())

trace_table(demo_trace).head(8) if pd is not None else trace_table(demo_trace)[:8]
```

```
message bytes = 28
digest        = 4fd9e99c31139dad682094dc67a3a9d6d27a3b64fe30fe79d579c44e149fd75e
hashlib match = True
```

Out[22]:

	t	W[t]	T1	T2	a_out	e_out
0	0	4e455855	41bd45bd	08909ae5	4a4de0a2	e70d3af7
1	1	537c464f	454165c9	0a518a15	4f92efde	81b0593b
2	2	4c447c48	7f7365c9	f0b3a544	70270b0d	3adb144e
3	3	4f4f4b7c	2c4dc489	e25661c9	0ea42652	9657aaf0
4	4	53434152	3f397c04	704663ed	af7fdff1	89875ca6
5	5	7c524554	d7ac3845	99545a10	71009255	273f2823
6	6	55524e7c	cb207b36	fbc78fa6	c6e80adc	3b478643
7	7	80000000	cc44b9e0	aedfe1c1	7b249ba1	dae8e032

In [23]:
```
single_block_msg = b"NEXUS|HOOK|SCAR|RETURN|PHASE|ECHO|SHIFT|GLASS|KEY|H=PI/9"[:55]
single_trace = sha256_trace(single_block_msg)
single_rev = reverse_block_no_trace(
    single_trace.digest_hex,
    padded_block64=pad_sha256(single_block_msg),
)

print("single-block bytes =", len(single_block_msg))
print("single-block digest =", single_trace.digest_hex)
print("tail state          =", tail_state_from_digest(single_trace.digest_hex))
print("reverse verified    =", single_rev.verified)
print("recovered start == IV =", single_rev.recovered_start_state == tuple(STD_H))
```

```
single-block bytes = 55
single-block digest = 01dfa391badd78914b59a61a1b982a1d9ddd874a8a175279dc83dd325c93b148
tail state          = {'a63': '97d5bd2a', 'b63': 'ff75ca0c', 'c63': '0eeab2a8', 'd63': '764834e3', 'e63': '4ccf34cb',
'f63': 'ef11e9ed', 'g63': 'bd000387', 'h63': '00b2e42f'}
reverse verified    = True
recovered start == IV = True
```

In [24]:
```
comparison = run_family_comparison()
comparison
```

Out[24]:

	name	bytes	blocks	std_digest	
0	K_constants_full	256	5	e4c62ae41929873b99fa3f871694451c9dbd0300448aba...	5f80e54690df0f0cfb86a07823db4ff973
1	K_constants_half	128	3	d2c917c55c86db7769a169d0ee5b8bf63428307d11168e...	3684b8cc69bb55982ddc7e6540d064b0ea
2	H_constants	32	1	5cce668898a2e2d7dedbb552165a2a77596d092b03df3f...	a79262642290e355c68887896a2aa04337
3	K_INV_as_msg	256	5	25128c8cc6689732e068c5209f2ec3bf3d8ce122617f5a...	e5748049c125a3491d00453b562f094be7
4	K_phase_shifted	512	9	eee018930c2cb7aacacc6ff72b792dcc067ff5ac6673a2...	4949402d012aecebb6cae0a713aece2b2c
5	K_xor_reverse	256	5	8b8e219f0d987c0a578da684124fae28a4940f7f8de30a...	8af3916b93dee79af6ddb617c660d11a28
6	sigma_feedback	128	3	8bd5c310f00358d172a78ebef80ae744114152dab33fc0...	9b453ec1a80ec2e77dd5ccae901b3b768a
7	structured_harmonic	480	8	23db38ca7ddab7ffc2d531f4722c4818ab4ff1d1368a8a...	70a47abf3599936f25bfb598194c0bd1e

In [25]:

```
biomarkers = {
    "mark1_geometry": mark1_geometry(),
    "d_anomaly_count": d_anomaly_count(STD_K),
    "constant_digit_histogram": hex_digit_hist_words(STD_K),
    "prime_root_fraction_stats": prime_root_fraction_stats(),
}
biomarkers
```

Out[25]:

```
{'mark1_geometry': {'H_radians': 0.3490658503988659,
  'H_degrees': 20.0,
  'arc_length': 0.3490658503988659,
  'chord_length': 0.34729635533386066,
  'relative_loss': 0.005069229954701356,
  'steps_to_circle': 18},
 'd_anomaly_count': 19,
 'constant_digit_histogram': {'0': 31,
  '1': 38,
  '2': 33,
  '3': 25,
  '4': 32,
  '5': 31,
  '6': 31,
  '7': 36,
  '8': 38,
  '9': 31,
  'a': 36,
  'b': 34,
  'c': 43,
  'd': 19,
  'e': 25,
  'f': 29},
 'prime_root_fraction_stats': {'mean_fraction': 0.4777901391168488,
  'median_fraction': 0.46800936127245096,
  'std_fraction': 0.262102258394429,
  'mean_abs_to_H': 0.2455165736257380 3}}
```


In [26]:
```
tri_df = tri_channel_trace(single_trace)
print("carry exhaust bits (block 0) =", carry_exhaust_bits(single_trace, 0))
tri_df.head(10) if pd is not None else tri_df[:10]
```

```
carry exhaust bits (block 0) = 518
```

Out[26]:

	t	carry_pop	xor	carry	and2	sum
0	0	6	492ddf58	009000a5	0120014a	4a4de0a2
1	1	4	4f10eddc	00410201	00820402	4f92f1de
2	2	9	62a0ff08	805b00c4	00b60188	63570090
3	3	6	cbafa847	000053a0	0000a740	cbb04f87
4	4	14	66246740	995a90a3	32b52146	98d98886
5	5	6	3de937b9	4210c004	84218008	c20ab7c1
6	6	8	c14ce9a5	2ea00210	5d400420	1e8cedc5
7	7	8	bb7b560b	04848930	09091260	c484686b
8	8	11	c0bd170f	2a4268e0	5484d1c0	1541e8cf
9	9	7	a9c463e7	10109418	20212830	c9e58c17

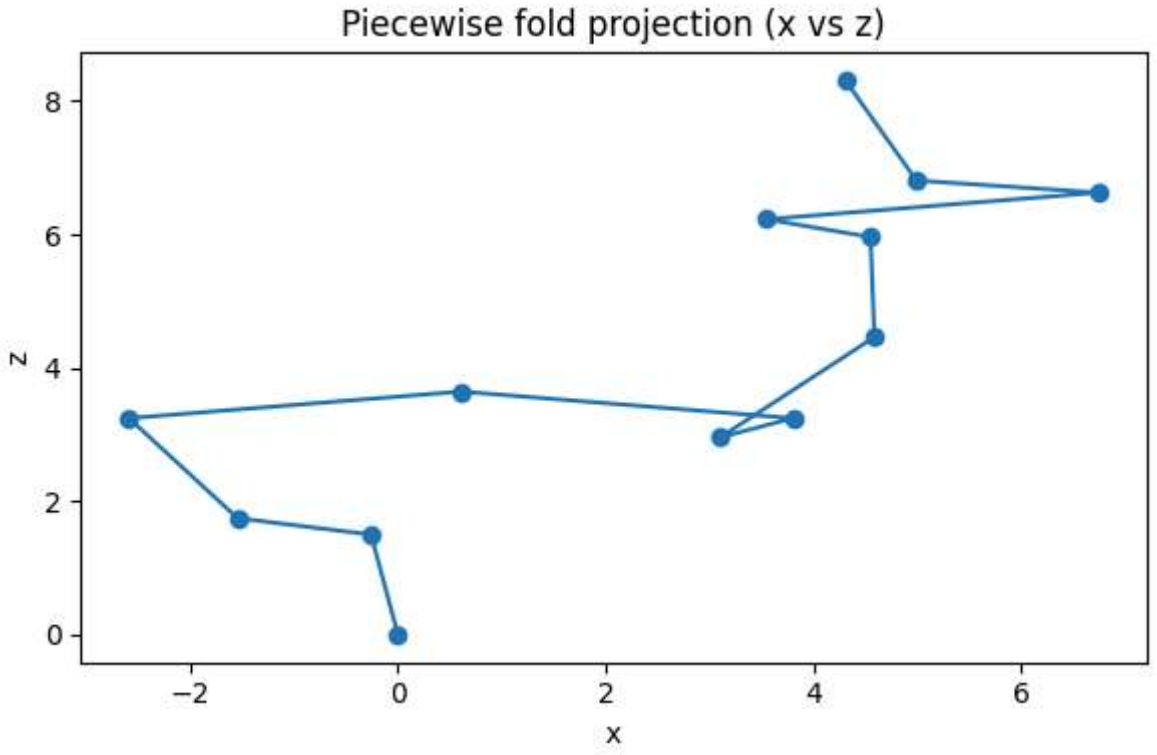
In [27]:
```
print("first 32 BBP hex digits of π =", bbp_hex_window(0, 32))
print("basin path for first 32 digits =", bbp_basins(0, 32))
```

```
first 32 BBP hex digits of π = 243F6A8885A308D313198A2E03707344
basin path for first 32 digits = [3, 1, 3, 3, 1, 2, 1, 1, 1, 2, 2, 3, 3, 1, 1, 3, 1, 3, 1, 2, 1, 2, 3, 1, 3, 3, 1, 3,
1, 3, 1, 1]
```

In [28]:
```
sequence = "ACDEFGHIKLMNPQRSTVWY"
signal = seq_to_signal(sequence)
labels = "HCHBBCHHCBCH"
coords, fold_metrics = piecewise_fold(labels)

print("sequence acf lag-3 =", acf_lag(signal, lag=3))
print("z_sarrus =", compute_z_sarrus(sequence))
print("piecewise fold metrics =", fold_metrics)

if plt is not None:
    plt.figure(figsize=(6, 4))
    plt.plot(coords[:, 0], coords[:, 2], marker="o")
    plt.title("Piecewise fold projection (x vs z)")
    plt.xlabel("x")
```

```
    plt.ylabel("z")
    plt.tight_layout()
    plt.show()
```

```
sequence acf lag-3 = 0.11848712231149425
z_sarrus = 0.9607843137254901
piecewise fold metrics = {'points': 13, 'radius_of_gyration': 3.8955183897231294, 'z_span': 8.3073060636057}
```



In [29]:
```
multi_msg = harmonic_message_family()["structured_harmonic"]
print(last_block_boundary_diagnostic(multi_msg))
print("schedule consistency on last block =", schedule_consistency(sha256_trace(multi_msg).schedule[-1]))
print("delta attraction example =", delta_attraction(delta=1.25, tau=0.8))
```

```
{'blocks': 8, 'last_block_chaining_matches_prev': True, 'last_block_input_state': ['5f4cacc8', 'f7378d0c', '5764298
0', 'c1315ef1', 'c32755a1', '74cf4247', '5e939798', 'cdea6666'], 'derived_from_prev': ['5f4cacc8', 'f7378d0c', '57642
980', 'c1315ef1', 'c32755a1', '74cf4247', '5e939798', 'cdea6666']}
schedule consistency on last block = {'consistent': True, 'nonzero_errors': 0}
delta attraction example = {'S': 0.5454153912482279, 'regime': 'damping'}
```

10. Working note

This notebook is designed as a computational appendix. The functions are intended to be imported, rerun, or extended directly from the paper context.

In [30]:
```
import math
import struct
import hashlib
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (9, 4)
plt.rcParams["axes.grid"] = True

M32 = 0xFFFFFFFF
H = math.pi / 9
PHI = (1 + math.sqrt(5)) / 2
EULER = math.e

STD_H = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

STD_K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]
```

```python
def popcount(x: int) -> int:
    return int(x).bit_count()

def rotr(x: int, n: int) -> int:
    return ((x >> n) | (x << (32 - n))) & M32

def Ch(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (~x & z)) & M32

def Maj(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (x & z) ^ (y & z)) & M32

def Sig0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sig1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sig0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sig1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def pad_sha256(msg: bytes) -> bytes:
    ml = len(msg) * 8
    out = bytearray(msg)
    out.append(0x80)
    while (len(out) % 64) != 56:
        out.append(0)
    out += ml.to_bytes(8, "big")
    return bytes(out)

def words_from_hex(h: str):
    return [int(h[i:i+8], 16) for i in range(0, len(h), 8)]

print({
    "H": H,
    "phi": PHI,
    "e": EULER,
    "sha_constants": len(STD_K),
```

```
    "iv_words": len(STD_H),
})
```

```
{'H': 0.3490658503988659, 'phi': 1.618033988749895, 'e': 2.718281828459045, 'sha_constants': 64, 'iv_words': 8}
```

I. Ontological inversion and the variable-shape frame

The paper treats the computational substrate as primary and the physical carrier as adapter. A variable is not an empty box; it is a constraint mold. A state is not merely assigned; it is forced into lawful fit.

The code below pulls the central geometric anchors into executable form:

- the **Mark 1 attractor** (H=\pi/9),
- the **H-triangle scale** where height equals the angle,
- the **48D width** as (6\times 8),
- and the first-order braid / scar functions used across the project corpus.

In [31]:
```
def h_triangle_scale(theta: float) -> float:
    return theta / math.sin(theta)

def braid_step(T1: int, T2: int, modulus: int = 2**32) -> int:
    return (T1 + T2) % modulus

def scar(a: int, b: int) -> int:
    return popcount(a ^ b)

L9 = h_triangle_scale(H)
hex_tiling = 6
byte_width = 8
cpu_width = hex_tiling * byte_width

df_geom = pd.DataFrame({
    "quantity": ["H = π/9", "L9 = H/sin(H)", "hex_tiling", "byte_width", "cpu_width"],
    "value": [H, L9, hex_tiling, byte_width, cpu_width]
})
df_geom
```

Out[31]:

	quantity	value
0	H = π/9	0.349066
1	L9 = H/sin(H)	1.020600
2	hex_tiling	6.000000
3	byte_width	8.000000
4	cpu_width	48.000000

II. Universal ROM and BBP-style addressing

A major claim in the paper is that the substrate behaves as if it contains a pre-encoded read-only manifold, where direct addressing matters more than linear generation.

The following BBP implementation is a **direct hexadecimal digit reader** for (\pi).
It is included here not as a statement about metaphysics, but as an executable addressing primitive aligned to the notebook's Universal ROM section.

In [32]:
```
def _bbp_series_term(j: int, n: int) -> float:
    s = 0.0
    for k in range(n + 1):
        s = (s + pow(16, n - k, 8 * k + j) / (8 * k + j)) % 1.0
    k = n + 1
    eps = 1e-17
    while True:
        term = (16.0 ** (n - k)) / (8 * k + j)
        if term < eps:
            break
        s += term
        k += 1
    return s

def pi_hex_digit(n: int) -> int:
    x = (
        4 * _bbp_series_term(1, n)
        - 2 * _bbp_series_term(4, n)
```

```
        - 1 * _bbp_series_term(5, n)
        - 1 * _bbp_series_term(6, n)
    ) % 1.0
    return int(16 * x)

hex_digits = "".join("0123456789abcdef"[pi_hex_digit(i)] for i in range(16))
print("first 16 hex digits of π after decimal point:", hex_digits)
```

```
first 16 hex digits of π after decimal point: 243f6a8885a308d3
```

III. Mark 1 attractor, harmonic stabilization, and the nonagonal closure floor

The paper treats (H=\pi/9) as the governing stability band and links it to 18-step closure, duty-cycle regulation, and feedback correction.

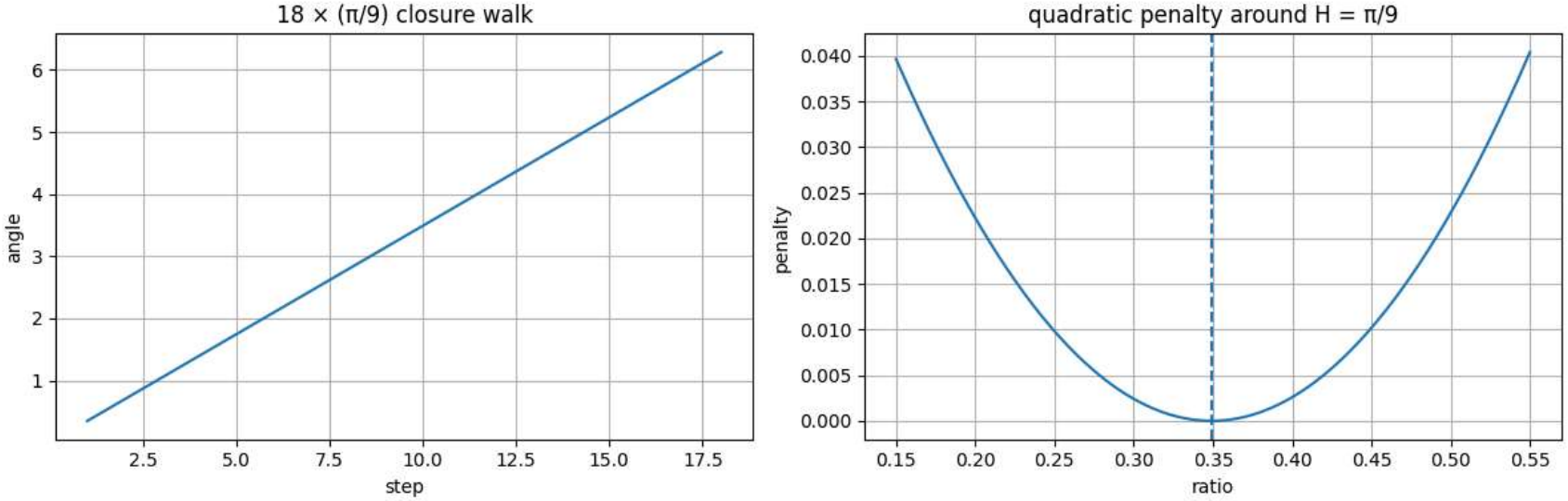
This section instruments those pieces numerically:

- 18 closure steps around (H),
- the 35/65 corridor,
- and a simple harmonic error scan around the attractor.

In [33]:
```
theta_steps = np.arange(1, 19) * H
closure_error = np.abs((18 * H) - (2 * math.pi))

scan = np.linspace(0.15, 0.55, 300)
stability_penalty = (scan - H) ** 2

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(np.arange(1, 19), theta_steps)
ax[0].set_title("18 × (π/9) closure walk")
ax[0].set_xlabel("step")
ax[0].set_ylabel("angle")

ax[1].plot(scan, stability_penalty)
ax[1].axvline(H, linestyle="--")
ax[1].set_title("quadratic penalty around H = π/9")
ax[1].set_xlabel("ratio")
ax[1].set_ylabel("penalty")
```

```
plt.tight_layout()
print({"18H_minus_2pi": closure_error, "H": H, "1-H": 1-H})
plt.show()
```

```
{'18H_minus_2pi': np.float64(0.0), 'H': 0.3490658503988659, '1-H': 0.650934149601134}
```



IV. The tri-transistor echo and ternary basin mapping

The paper's hardware claim is not "three discrete switches" but a **single carrier wave moving through three basin states**:

- **Basin 0** → source / cold base / uncoupled potential
- **Basin 1** → oxide gap / carry / work
- **Basin 2** → drain / sum / actualized residue

The next cell makes this explicit through finite-register addition, using the irreducible triad:

[a+b = (a \oplus b) + 2(a \wedge b)]

and maps that triad to the transistor framing.

In [34]:
```
def kernel_add32(a: int, b: int):
    xor_term = (a ^ b) & M32
```

```
    carry_term = ((a & b) << 1) & M32
    sum_term = (xor_term + carry_term) & M32
    return xor_term, carry_term, sum_term

def tri_basin_map(a: int, b: int):
    xor_term, carry_term, sum_term = kernel_add32(a, b)
    return {
        "input_a": f"0x{a:08x}",
        "input_b": f"0x{b:08x}",
        "basin_0_xor": f"0x{xor_term:08x}",
        "basin_1_carry": f"0x{carry_term:08x}",
        "basin_2_sum": f"0x{sum_term:08x}",
        "xor_popcount": popcount(xor_term),
        "carry_popcount": popcount(carry_term),
        "sum_popcount": popcount(sum_term),
    }

examples = [
    (0x243F6A88, 0x6A09E667),
    (0xB7E15162, 0x6A09E667),
    (0x9E3779B9, 0x6A09E667),
    (0xB2D70000, 0x6A09E667),
]
tri_df = pd.DataFrame([tri_basin_map(a, b) for a, b in examples])
tri_df
```

Out[34]:

	input_a	input_b	basin_0_xor	basin_1_carry	basin_2_sum	xor_popcount	carry_popcount	sum_popcount
0	0x243f6a88	0x6a09e667	0x4e368cef	0x4012c400	0x8e4950ef	18	6	16
1	0xb7e15162	0x6a09e667	0xdde8b705	0x440280c4	0x21eb37c9	18	7	17
2	0x9e3779b9	0x6a09e667	0xf43e9fde	0x1402c042	0x08416020	22	7	6
3	0xb2d70000	0x6a09e667	0xd8dee667	0x44020000	0x1ce0e667	20	3	16

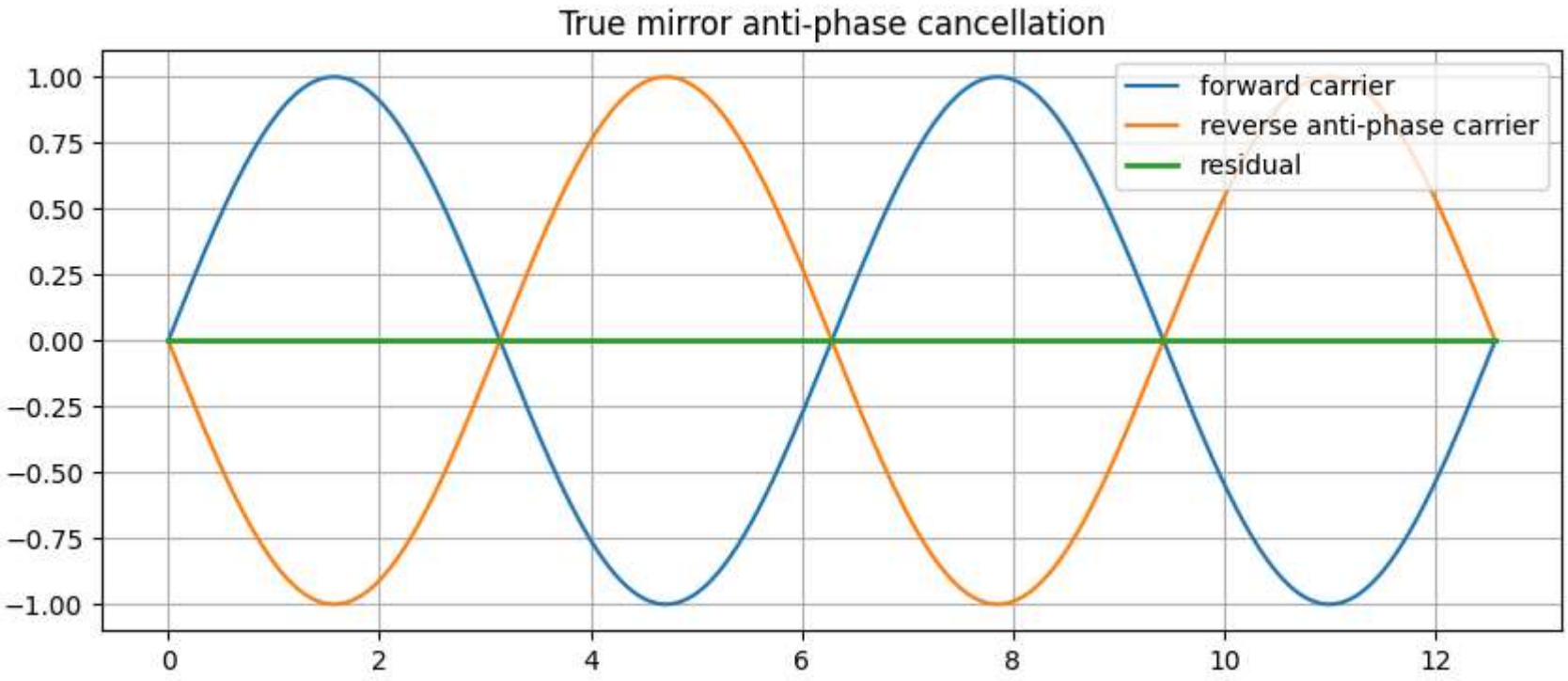
V. True mirror anti-phase cancellation

The paper's "True Mirror" section describes a forward carrier and a reverse anti-phase carrier passing through the same channel and nullifying deflection at the boundary.

The following is a direct signal-level toy model of that mechanism:

- forward wave,
- reverse anti-phase wave,
- residual after superposition.

In [35]:
```
t = np.linspace(0, 4*np.pi, 800)
forward = np.sin(t)
reverse = -np.sin(t)
residual = forward + reverse

energy_forward = float(np.mean(forward**2))
energy_residual = float(np.mean(residual**2))

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
ax.plot(t, forward, label="forward carrier")
ax.plot(t, reverse, label="reverse anti-phase carrier")
ax.plot(t, residual, label="residual", linewidth=2)
ax.set_title("True mirror anti-phase cancellation")
ax.legend()
plt.show()

print({"forward_energy": energy_forward, "residual_energy": energy_residual})
```




```
{'forward_energy': 0.499375, 'residual_energy': 0.0}
```

VI. Dual-wave ontology and the Pythagorean storage law

The notebook now shifts from the transistor face to the storage law that the paper uses to unify computation, physics, and folding:

[E_{total} = \sqrt{V^2 + S^2}]

where:

- (V) is the explicit **value channel**
- (S) is the latent **shape / residue channel**

Below, the finite-register kernel is measured across a spread of inputs so the value / shape split can be inspected directly.

In [36]:
```
def dual_wave_metrics(a: int, b: int):
    xor_term, carry_term, sum_term = kernel_add32(a, b)
```

```
    V = popcount(xor_term)
    S = popcount(carry_term)
    E_total = math.sqrt(V*V + S*S)
    return {
        "a": f"0x{a:08x}",
        "b": f"0x{b:08x}",
        "V_pop": V,
        "S_pop": S,
        "sum_pop": popcount(sum_term),
        "E_total": E_total,
    }

pairs = [
    (STD_H[0], STD_H[1]),
    (STD_H[2], STD_H[3]),
    (STD_K[0], STD_K[1]),
    (STD_K[34], STD_K[35]),
    (0xAAAAAAAA, 0x55555555),
    (0xFFFFFFFF, 0x00000001),
]
dw_df = pd.DataFrame([dual_wave_metrics(a, b) for a, b in pairs])
dw_df
```

Out[36]:

	a	b	V_pop	S_pop	sum_pop	E_total
0	0x6a09e667	0xbb67ae85	15	10	15	18.027756
1	0x3c6ef372	0xa54ff53a	10	14	18	17.204651
2	0x428a2f98	0x71374491	17	5	15	17.720045
3	0x4d2c6dfc	0x53380d13	15	8	15	17.000000
4	0xaaaaaaaa	0x55555555	32	0	32	32.000000
5	0xffffffff	0x00000001	31	1	0	31.016125

VII. SHA-256 as a mechanical mold

This notebook uses the paper's SHA-facing side heavily.

The next cell defines a compact but complete forward trace instrumenter for SHA-256:

- message padding,
- schedule expansion,
- 64-round trace,
- per-round (T_1), (T_2), carry exhaust,
- final digest.

This is the executable hinge for later sections on:

- the `'d'` anomaly,
- Glass Key behavior,
- reverse walk,
- substrate parity.

In [37]:
```python
@dataclass
class SHATrace:
    digest_hex: str
    rounds: list
    schedule: list
    padded_len: int

def sha256_trace(msg: bytes):
    padded = pad_sha256(msg)
    block = padded[:64]   # single-block focus for instrumentation cells
    W = list(struct.unpack(">16I", block))
    for t in range(16, 64):
        W.append((sig1(W[t-2]) + W[t-7] + sig0(W[t-15]) + W[t-16]) & M32)

    a, b, c, d, e, f, g, h = STD_H[:]
    rounds = []

    for t in range(64):
        state_term = (h + Sig1(e) + Ch(e, f, g)) & M32
        T1 = (state_term + STD_K[t] + W[t]) & M32
        T2 = (Sig0(a) + Maj(a, b, c)) & M32
        carry_t1 = ((state_term & ((STD_K[t] + W[t]) & M32)) << 1) & M32

        a_next = (T1 + T2) & M32
```

```
        e_next = (d + T1) & M32

        rounds.append({
            "t": t,
            "a": a, "b": b, "c": c, "d": d, "e": e, "f": f, "g": g, "h": h,
            "W": W[t], "K": STD_K[t], "STATE": state_term, "T1": T1, "T2": T2,
            "carry_T1": carry_t1,
            "a_after": a_next, "e_after": e_next,
        })

        h, g, f, e, d, c, b, a = g, f, e, e_next, c, b, a, a_next

    H_out = [((x + y) & M32) for x, y in zip(STD_H, [a, b, c, d, e, f, g, h])]
    digest_hex = "".join(f"{x:08x}" for x in H_out)
    return SHATrace(digest_hex=digest_hex, rounds=rounds, schedule=W, padded_len=len(padded))

demo_msg = b"NEXUS|TRANSISTOR|UNIVERSAL|SUBSTRATE|"
demo_trace = sha256_trace(demo_msg)
print({"digest": demo_trace.digest_hex, "rounds": len(demo_trace.rounds), "padded_len": demo_trace.padded_len})
```

```
{'digest': '449d2c6c25511889790ae0745abc7a6ed26810f2306393d83e64f82bd9a3597e', 'rounds': 64, 'padded_len': 64}
```

VIII. The `'d'` anomaly and the 75% duty-cycle heartbeat

The paper emphasizes a specific empirical anomaly: the hexadecimal digit `'d'` (binary `1101`) appearing at elevated frequency in the SHA constants.

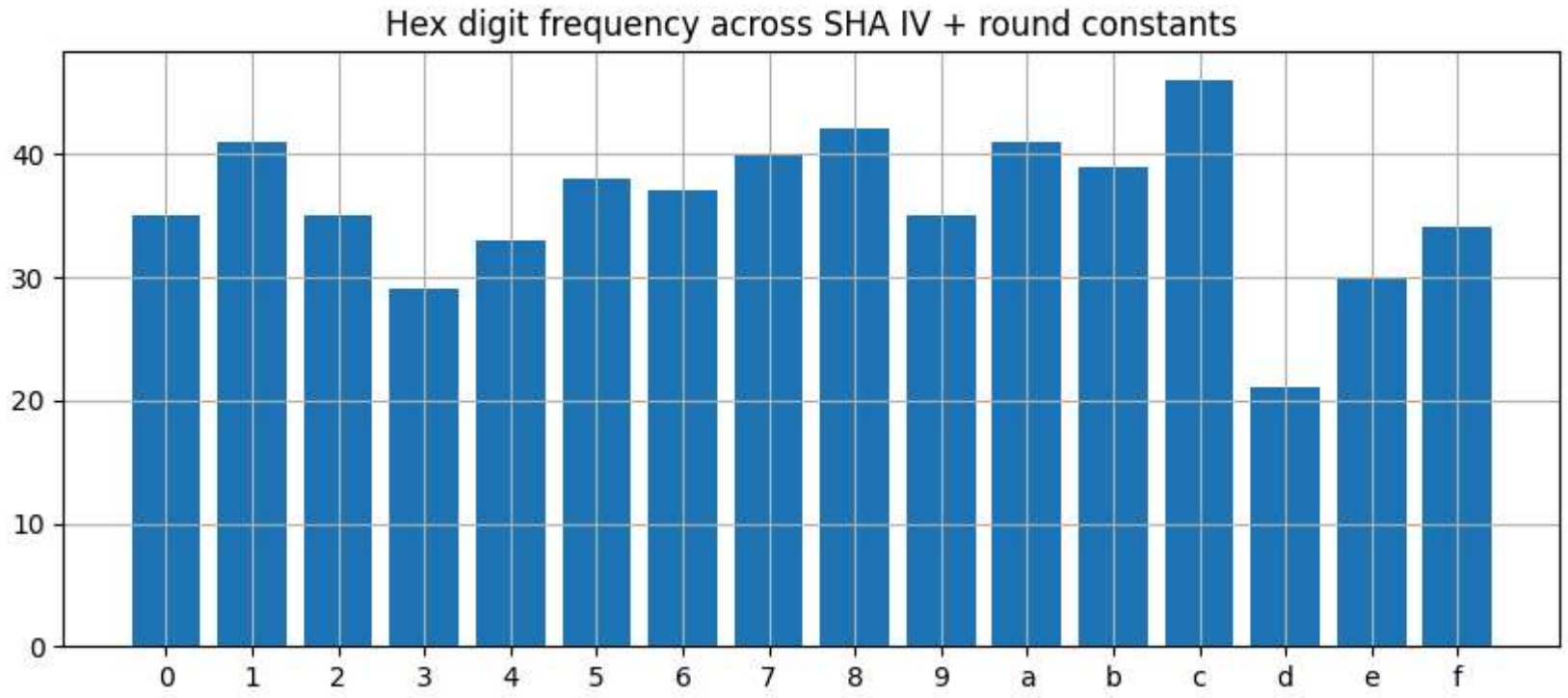
This section counts the hexadecimal digits of:

- the 64 round constants,
- the 8 IV words,
- the combined field, and highlights the `'d'` count directly.

In [38]:
```
def hex_digit_hist(words):
    h = "".join(f"{w:08x}" for w in words)
    counts = {ch: h.count(ch) for ch in "0123456789abcdef"}
    return pd.Series(counts).sort_index()

hist_k = hex_digit_hist(STD_K)
```

```
hist_h = hex_digit_hist(STD_H)
hist_all = hist_k.add(hist_h, fill_value=0).astype(int)

freq_df = pd.DataFrame({
    "K_constants": hist_k,
    "H_constants": hist_h,
    "combined": hist_all,
})
display(freq_df.T)

fig, ax = plt.subplots(figsize=(10,4))
ax.bar(freq_df.index, freq_df["combined"])
ax.set_title("Hex digit frequency across SHA IV + round constants")
plt.show()

print({
    "d_count_K": int(hist_k["d"]),
    "d_count_H": int(hist_h["d"]),
    "d_count_combined": int(hist_all["d"]),
})
```

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
K_constants	31	38	33	25	32	31	31	36	38	31	36	34	43	19	25	29
H_constants	4	3	2	4	1	7	6	4	4	4	5	5	3	2	5	5
combined	35	41	35	29	33	38	37	40	42	35	41	39	46	21	30	34




```
{'d_count_K': 19, 'd_count_H': 2, 'd_count_combined': 21}
```

IX. Harmonic SHA reflection and structured-input probing

The project corpus repeatedly tests whether certain structured inputs "lock" more cleanly to the SHA mechanical mold than arbitrary strings.

The next cell builds a family of harmonic / resonant inputs and compares their digests against:

- the IV field,
- the K field,
- and a few direct reversals.

This is not a preimage solver; it is a **geometry-probing scan**.

In [39]:
```
def reverse_hex_variants(hex_digest: str):
    bytes_ = [hex_digest[i:i+2] for i in range(0, len(hex_digest), 2)]
```

```
    words = [hex_digest[i:i+8] for i in range(0, len(hex_digest), 8)]

    def reverse_bits_in_byte(hx):
        b = int(hx, 16)
        rb = int(f"{b:08b}"[::-1], 2)
        return f"{rb:02x}"

    return {
        "chars_rev": hex_digest[::-1],
        "bytes_rev": "".join(reversed(bytes_)),
        "words32_rev": "".join(reversed(words)),
        "bits_each_byte_rev": "".join(reverse_bits_in_byte(x) for x in bytes_),
    }

def harmonic_inputs():
    motifs = [
        "NEXUS|FOLD|HOOK|RETURN|",
        "TRANSISTOR|BASE|EMITTER|COLLECTOR|",
        "VALUE|SHAPE|MIRROR|",
        "PI|PHI|E|ROM|",
        "TRI|CHANNEL|ABI|",
    ]
    out = []
    for i in range(10):
        m = motifs[i % len(motifs)]
        r = i % len(m)
        rot = m[r:] + m[:r]
        out.append((f"h{i:02d}", (rot + rot[::-1] + f"|{i*i:04d}|").encode()))
    return out

scan_rows = []
for name, msg in harmonic_inputs():
    d = hashlib.sha256(msg).hexdigest()
    vars_ = reverse_hex_variants(d)
    scan_rows.append({
        "name": name,
        "bytes": len(msg),
        "digest": d[:16] + "...",
        "digest_pop": popcount(int(d, 16)),
        "chars_rev_pop": popcount(int(vars_["chars_rev"], 16)),
        "words32_rev_pop": popcount(int(vars_["words32_rev"], 16)),
    })
```

```
harm_df = pd.DataFrame(scan_rows)
harm_df
```

Out[39]:

	name	bytes	digest	digest_pop	chars_rev_pop	words32_rev_pop
0	h00	52	41898e022833fc6b...	124	124	124
1	h01	74	61dfdd5e77b728cc...	134	134	134
2	h02	44	98453336a4646ac0...	128	128	128
3	h03	32	fad50ed11bf63a34...	136	136	136
4	h04	38	cab130eca9e2b6f3...	132	132	132
5	h05	52	49896ca831bb619b...	125	125	125
6	h06	74	837d0f91a8f95b71...	133	133	133
7	h07	44	2076ce01dc776580...	125	125	125
8	h08	32	a45b5ad1edfd9490...	127	127	127
9	h09	38	a5e28017442e2397...	125	125	125

X. Exact backward peel for a known schedule

To keep the notebook grounded, it includes the exact **single-block reverse peel** that was already stabilized in the project:

- start from the digest,
- subtract feed-forward,
- peel rounds backward algebraically,
- no saved forward trace.

This is the direct reverse seam for the paper's reversibility sections.

In [40]:

```
def digest_words(digest_hex: str):
    return [int(digest_hex[i:i+8], 16) for i in range(0, 64, 8)]
```

```python
def subtract_feedforward(digest_hex: str, H_in=STD_H):
    dw = digest_words(digest_hex)
    return tuple(((d - h) & M32) for d, h in zip(dw, H_in))

def expand_schedule_from_block(block64: bytes):
    W = list(struct.unpack(">16I", block64))
    for t in range(16, 64):
        W.append((sig1(W[t-2]) + W[t-7] + sig0(W[t-15]) + W[t-16]) & M32)
    return W

def reverse_round_no_trace(after_state, Wt, Kt):
    a1, b1, c1, d1, e1, f1, g1, h1 = after_state
    a0 = b1
    b0 = c1
    c0 = d1
    e0 = f1
    f0 = g1
    g0 = h1
    T2 = (Sig0(a0) + Maj(a0, b0, c0)) & M32
    T1 = (a1 - T2) & M32
    d0 = (e1 - T1) & M32
    h0 = (T1 - Sig1(e0) - Ch(e0, f0, g0) - Kt - Wt) & M32
    return (a0, b0, c0, d0, e0, f0, g0, h0)

def reverse_block_no_trace(digest_hex: str, padded_block64: bytes, H_in=STD_H, K=STD_K):
    W = expand_schedule_from_block(padded_block64)
    cur = subtract_feedforward(digest_hex, H_in)
    states = [cur]
    for t in range(63, -1, -1):
        cur = reverse_round_no_trace(cur, W[t], K[t])
        states.append(cur)
    return {
        "final_working_state": states[0],
        "recovered_start_state": cur,
        "verified": tuple(H_in) == cur,
        "states_backward": states,
    }

one_block_msg = b"TRI|TRANSISTOR|ECHO|TRUE|MIRROR|"
one_block_digest = hashlib.sha256(one_block_msg).hexdigest()
one_block_padded = pad_sha256(one_block_msg)
reverse_result = reverse_block_no_trace(one_block_digest, one_block_padded)
```

```
print({
    "digest": one_block_digest,
    "verified": reverse_result["verified"],
    "recovered_start_state_matches_iv": reverse_result["recovered_start_state"] == tuple(STD_H),
})
```

```
{'digest': '3f288f2f7fa9298736c153c640178461d9de7c064fdafa1b9dbec024d46f583b', 'verified': True, 'recovered_start_sta
te_matches_iv': True}
```

XI. Samson backpressure as a synthetic controller

The paper uses Samson backpressure as a stabilization narrative and as the navigation logic for reverse search through high-friction regions.

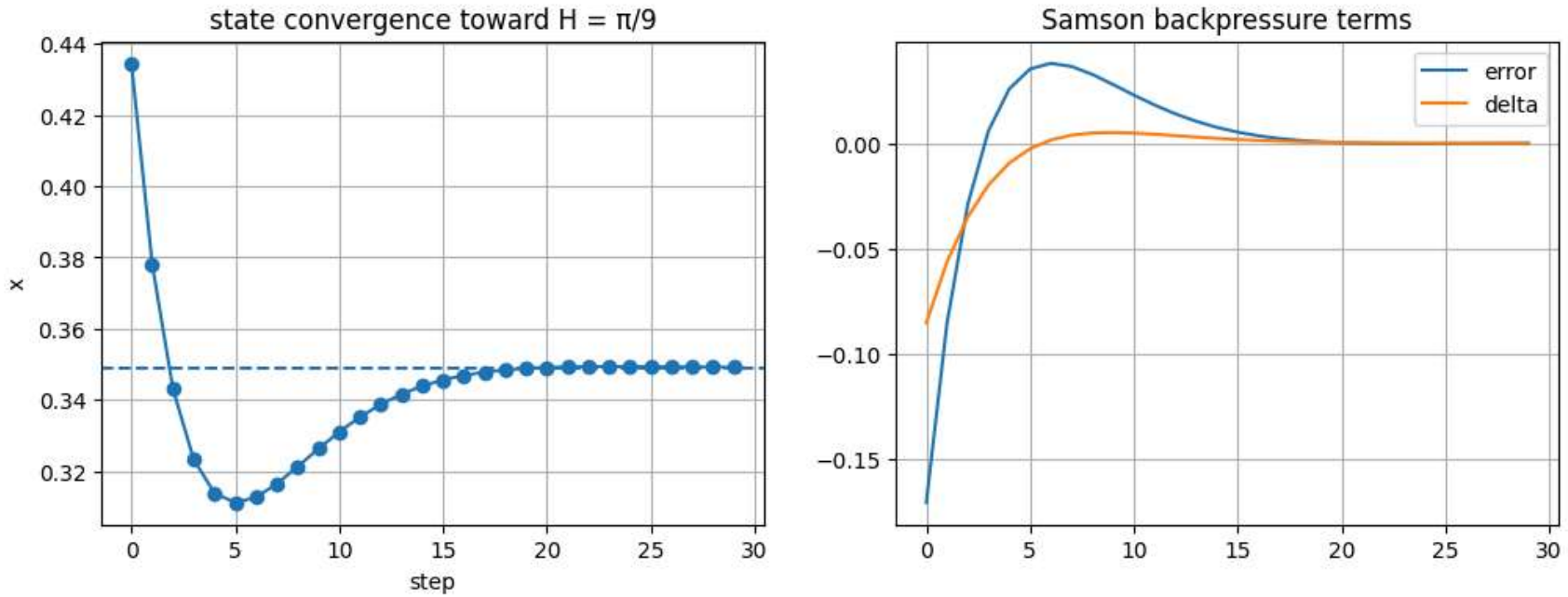
The next cell provides a synthetic controller that:

- starts away from the Mark 1 attractor,
- applies proportional / integral style correction,
- converges toward (H=\pi/9).

This gives an executable picture of “stay in the waist” dynamics.

In [41]:
```
class SamsonController:
    def __init__(self, target=H, kp=0.42, ki=0.08):
        self.target = float(target)
        self.kp = float(kp)
        self.ki = float(ki)
        self.integral = 0.0

    def step(self, x):
        err = self.target - x
        self.integral += err
        delta = self.kp * err + self.ki * self.integral
        return x + delta, err, delta

controller = SamsonController()
x = 0.52
xs, errs, deltas = [], [], []
for _ in range(30):
```

```
    x, err, delta = controller.step(x)
    xs.append(x)
    errs.append(err)
    deltas.append(delta)

fig, ax = plt.subplots(1, 2, figsize=(12,4))
ax[0].plot(xs, marker="o")
ax[0].axhline(H, linestyle="--")
ax[0].set_title("state convergence toward H = π/9")
ax[0].set_xlabel("step")
ax[0].set_ylabel("x")

ax[1].plot(errs, label="error")
ax[1].plot(deltas, label="delta")
ax[1].set_title("Samson backpressure terms")
ax[1].legend()
plt.show()
```



XII. Silicon / carbon parity and a protein-folding style renderer

The transistor paper explicitly leans on substrate parity:

- silicon constraint folding,
- carbon / protein constraint folding,
- same operator grammar.

The class below is a compact **A-ProteinFolding style renderer**:

- sequence → spectrum,
- spectrum → 3D coordinates,
- entropy, compression, and lag metrics.

This is not claiming biological completeness. It is an executable surrogate for the paper's silicon/carbon isomorphism section.

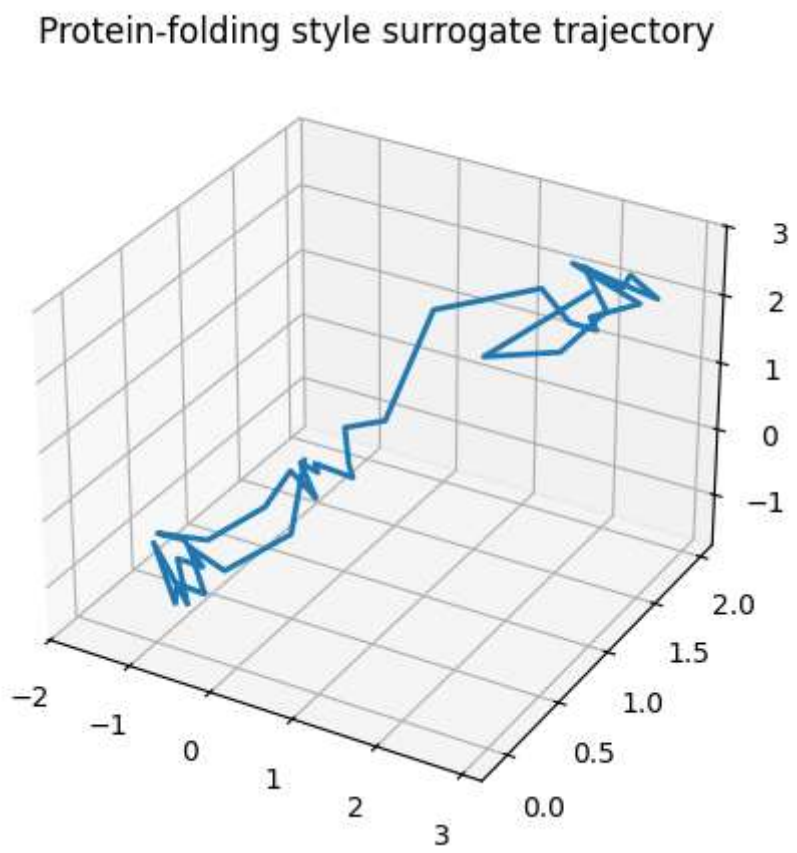
In [42]:
```
AA_HYDRO = {
    "A": 1.8, "C": 2.5, "D": -3.5, "E": -3.5, "F": 2.8, "G": -0.4,
    "H": -3.2, "I": 4.5, "K": -3.9, "L": 3.8, "M": 1.9, "N": -3.5,
    "P": -1.6, "Q": -3.5, "R": -4.5, "S": -0.8, "T": -0.7, "V": 4.2,
    "W": -0.9, "Y": -1.3
}
AA_CHARGE = {aa: 0.0 for aa in AA_HYDRO}
for aa in "KRH":
    AA_CHARGE[aa] = 1.0
for aa in "DE":
    AA_CHARGE[aa] = -1.0

class NexusFold:
    def __init__(self, H=H):
        self.H = H

    def sequence_to_spectrum(self, seq: str):
        seq = seq.upper()
        hydro = np.array([AA_HYDRO.get(a, 0.0) for a in seq], dtype=float)
        charge = np.array([AA_CHARGE.get(a, 0.0) for a in seq], dtype=float)
        phase = np.linspace(0, 2*np.pi*self.H*len(seq), len(seq), endpoint=False)
        signal = hydro + 1j*(charge + np.sin(phase))
        return signal

    def fold_protein(self, seq: str):
```

```
        spec = self.sequence_to_spectrum(seq)
        fx = np.fft.ifft(spec)
        fy = np.fft.ifft(spec * np.exp(1j*np.linspace(0, np.pi, len(spec))))
        fz = np.fft.ifft(spec * np.exp(1j*np.linspace(0, 2*np.pi, len(spec))))
        coords = np.vstack([fx.real, fy.real, fz.real]).T
        coords = np.cumsum(coords, axis=0)
        return coords, spec

    def calculate_spectral_entropy(self, spec):
        p = np.abs(spec)**2
        p = p / (p.sum() + 1e-12)
        return float(-(p * np.log2(p + 1e-12)).sum())

    def estimate_compression_ratio(self, seq: str, coords: np.ndarray):
        bbox = np.ptp(coords, axis=0).sum() + 1e-9
        return float(len(seq) / bbox)

    def structural_lag(self, coords: np.ndarray):
        diffs = np.diff(coords, axis=0)
        turn = np.linalg.norm(np.diff(diffs, axis=0), axis=1)
        return float(np.mean(turn)) if len(turn) else 0.0

fold_engine = NexusFold()
protein_seq = "MKWVTFISLLFLFSSAYSRGVFRRDAHKSEVAHRFKDLGE"
coords, spec = fold_engine.fold_protein(protein_seq)
bio_metrics = {
    "length": len(protein_seq),
    "spectral_entropy": fold_engine.calculate_spectral_entropy(spec),
    "compression_ratio": fold_engine.estimate_compression_ratio(protein_seq, coords),
    "structural_lag": fold_engine.structural_lag(coords),
}
bio_metrics
```

Out[42]:
```
{'length': 40,
 'spectral_entropy': 4.907392022970679,
 'compression_ratio': 3.5569214274147027,
 'structural_lag': 0.8641710675443438}
```

In [43]:
```
fig = plt.figure(figsize=(7,5))
ax = fig.add_subplot(projection='3d')
ax.plot(coords[:,0], coords[:,1], coords[:,2], linewidth=2)
```

```
ax.set_title("Protein-folding style surrogate trajectory")
plt.show()
```



XIII. Substrate parity table: SHA trace versus folding trace

The paper's strongest claim is not metaphor but operator identity across carriers.

Below, one SHA message trace and one folding trajectory are reduced into a common table:

- length,
- entropy,
- lag / carry style residue,
- attractor reference.

This is a compact substrate parity view.

In [44]:
```
sha_spec = np.array([r["T1"] ^ r["T2"] for r in demo_trace.rounds], dtype=np.uint64)
sha_entropy = float(pd.Series([popcount(int(x)) for x in sha_spec]).value_counts(normalize=True).pipe(lambda s: -(s*r
sha_lag = float(np.mean([popcount(r["carry_T1"]) for r in demo_trace.rounds]))

parity_df = pd.DataFrame([
    {
        "carrier": "SHA-256 single-block trace",
        "length_or_rounds": len(demo_trace.rounds),
        "entropy_proxy": sha_entropy,
        "lag_proxy": sha_lag,
        "anchor": H,
    },
    {
        "carrier": "Protein-folding surrogate",
        "length_or_rounds": len(protein_seq),
        "entropy_proxy": bio_metrics["spectral_entropy"],
        "lag_proxy": bio_metrics["structural_lag"],
        "anchor": H,
    },
])
parity_df
```

Out[44]:

	carrier	length_or_rounds	entropy_proxy	lag_proxy	anchor
0	SHA-256 single-block trace	64	3.420163	8.609375	0.349066
1	Protein-folding surrogate	40	4.907392	0.864171	0.349066

XIV. Larger appendix: code utilities collected into one place

The next cell is intentionally large. It consolidates a broad utility layer so the notebook can function as a real paper companion rather than a thin demo:

- digest reversal helpers,
- additive inverse helpers,

- topological pin scans,
- Glass Key candidate scans,
- transistor basin summaries,
- dual-wave summaries.

This appendix is meant to be read, modified, and extended.

In [45]:
```python
def additive_inverse32(x: int) -> int:
    return (-x) & M32

def digest_reverse_variants(hex_digest: str):
    bytes_ = [hex_digest[i:i+2] for i in range(0, len(hex_digest), 2)]
    words = [hex_digest[i:i+8] for i in range(0, len(hex_digest), 8)]
    def reverse_bits_in_byte(byte_hex: str) -> str:
        b = int(byte_hex, 16)
        rb = int(f"{b:08b}"[::-1], 2)
        return f"{rb:02x}"
    return {
        "chars_rev": hex_digest[::-1],
        "bytes_rev": "".join(reversed(bytes_)),
        "words32_rev": "".join(reversed(words)),
        "bits_each_byte_rev": "".join(reverse_bits_in_byte(x) for x in bytes_),
    }

def topological_pin_scan(msg: bytes):
    tr = sha256_trace(msg)
    rows = []
    for r in tr.rounds:
        rows.append({
            "t": r["t"],
            "W_pop": popcount(r["W"]),
            "K_pop": popcount(r["K"]),
            "T1_pop": popcount(r["T1"]),
            "T2_pop": popcount(r["T2"]),
            "carry_T1_pop": popcount(r["carry_T1"]),
        })
    df = pd.DataFrame(rows)
    df["pin_score"] = df["carry_T1_pop"] + np.abs(df["T1_pop"] - df["T2_pop"])
    return tr, df
```

```python
def glass_key_candidates(candidates):
    rows = []
    for name, msg in candidates:
        digest = hashlib.sha256(msg).hexdigest()
        vars_ = digest_reverse_variants(digest)
        rows.append({
            "name": name,
            "bytes": len(msg),
            "digest": digest,
            "digest_pop": popcount(int(digest, 16)),
            "chars_rev_pop": popcount(int(vars_["chars_rev"], 16)),
            "bytes_rev_pop": popcount(int(vars_["bytes_rev"], 16)),
            "words32_rev_pop": popcount(int(vars_["words32_rev"], 16)),
        })
    return pd.DataFrame(rows)

def basin_summary(words):
    rows = []
    for i in range(0, len(words)-1, 2):
        a, b = words[i], words[i+1]
        xor_term, carry_term, sum_term = kernel_add32(a, b)
        rows.append({
            "pair": f"{i}/{i+1}",
            "xor_pop": popcount(xor_term),
            "carry_pop": popcount(carry_term),
            "sum_pop": popcount(sum_term),
        })
    return pd.DataFrame(rows)

pin_trace, pin_df = topological_pin_scan(b"BASE|EMITTER|COLLECTOR|TRUE|MIRROR|")
display(pin_df.head(10))
display(glass_key_candidates(harmonic_inputs()).head(10))
display(basin_summary(STD_K[:16]))
```

	t	W_pop	K_pop	T1_pop	T2_pop	carry_T1_pop	pin_score
0	0	11	13	15	12	8	11
1	1	15	14	11	21	8	18
2	2	12	20	17	17	5	5
3	3	16	20	15	12	8	11
4	4	12	16	15	15	8	8
5	5	16	16	18	16	3	5
6	6	15	14	18	17	5	6
7	7	13	18	22	15	6	13
8	8	14	14	19	12	9	16
9	9	0	11	14	18	5	9

	name	bytes	digest	digest_pop	chars_rev_pop	bytes_rev_pop	words32_rev_pop
0	h00	52	41898e022833fc6bbe07ec9b8f0871c9519f547751d78a...	124	124	124	124
1	h01	74	61dfdd5e77b728cc9554e279a4a214446f49491a6b28f5...	134	134	134	134
2	h02	44	98453336a4646ac0c1e16a3b2ee46ff9d0f6ba13b05287...	128	128	128	128
3	h03	32	fad50ed11bf63a34bb3f930d1a3ccac95677a23ad4a477...	136	136	136	136
4	h04	38	cab130eca9e2b6f3a7cea07b4121ecad5df160fc86f2c4...	132	132	132	132
5	h05	52	49896ca831bb619b52ff96446cd81d96f830d68cd7e108...	125	125	125	125
6	h06	74	837d0f91a8f95b7170d935c522fdda6ee78ffc0d1041b5...	133	133	133	133
7	h07	44	2076ce01dc776580832c96a3d677a6b82309cb3091febe...	125	125	125	125
8	h08	32	a45b5ad1edfd94909a16127cea80aaf2f12fa97c5835f6...	127	127	127	127
9	h09	38	a5e28017442e239785d5798896268ac1ea82a552309b78...	125	125	125	125

	pair	xor_pop	carry_pop	sum_pop
0	0/1	17	5	15
1	2/3	14	12	21
2	4/5	16	8	15
3	6/7	16	7	19
4	8/9	15	5	15
5	10/11	20	5	14
6	12/13	15	11	19
7	14/15	20	6	20

XV. Closing section

This notebook does not try to prove every ontological statement in the paper. What it does provide is a **large executable substrate** aligned to the paper's actual sections:

- attractor geometry,
- BBP-style addressing,
- tri-transistor basin mechanics,
- anti-phase cancellation,
- dual-wave storage,
- SHA crystallization,
- anomaly measurement,
- reverse peeling,
- backpressure control,
- silicon/carbon parity.

It is meant to sit next to the paper as an operational companion.

In [46]:
```python
from __future__ import annotations

import ast
import json
import math
import re
import struct
import hashlib
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple, Iterable

import numpy as np
import pandas as pd
```

In [47]:
```python
H = math.pi / 9
M32 = 0xFFFFFFFF

STD_H = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

STD_K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]

INV_H = [(-x) & M32 for x in STD_H]
INV_K = [(-x) & M32 for x in STD_K]

PI_HEX_64 = '243F6A8885A308D313198A2E03707344A4093822299F31D0082EFA98EC4E6C89'

BBP_TRANS = {
    0: 2, 1: 4, 2: 8, 3: 15,
```

```
    4: 14, 5: 12, 6: 9, 7: 3,
    8: 7, 9: 13, 10: 10, 11: 5,
    12: 11, 13: 6, 14: 1, 15: 0,
}

EMBEDDED_BLOCK_MANIFEST = [{'block_id': 2, 'ok': True, 'title': '#!/usr/bin/env python3', 'lines': 493, 'defs': ['rot
```

1) Embedded code-block manifest

This section exposes the curated Python-block manifest directly inside the notebook.

It is intentionally **self-contained**:

- no external markdown file is read,
- the manifest is embedded as data,
- the notebook can be downloaded and executed as-is.

In [48]:
```
manifest = pd.DataFrame(EMBEDDED_BLOCK_MANIFEST)
manifest
```

Out[48]:

	block_id	ok	title	lines	defs	err
0	2	True	#!/usr/bin/env python3	493	[rotr, sigma0, sigma1, Sigma0, Sigma1, Ch, Maj...	
1	3	False	√2 as Algebraic ROM — Decompilation Results	50	[]	inva character (U+221 (<unknown li
2	4	True	import sympy as sp	29	[]	
3	5	True	import sympy as sp	6	[]	
4	6	True	import numpy as np	25	[]	
5	7	True	import sympy as sp	26	[]	
6	8	True	import sympy as sp	5	[]	
7	9	True	import sympy as sp	12	[]	
8	10	True	import sympy as sp	11	[]	
9	11	True	import sympy as sp	10	[]	
10	13	True	import sympy as sp	9	[]	
11	15	True	#!/usr/bin/env python3	741	[rotr, shr, ch, maj, sigma0, sigma1, gamma0, g...	
12	47	True	#!/usr/bin/env python3	504	[rotr, shr, ch, maj, sigma0, sigma1, gamma0, g...	
13	48	True	#!/usr/bin/env python3	642	[sha256_bytes, sha256_hex, bytes_to_words, wor...	
14	49	True	#!/usr/bin/env python3	741	[rotr, shr, ch, maj, sigma0, sigma1, gamma0, g...	
15	50	True	#!/usr/bin/env python3	619	[trace]	
16	51	True	"""	512	[S0, S1, Ch, Maj, s0, s1, sha_forward_trace, s...	

	block_id	ok	title	lines	defs	err
17	52	True	"""	319	[s0, s1, sha_n, A_shape, mean_A]	
18	53	True	"""	265	[s0, s1, full_trace, acf, bx]	
19	54	True	"""	247	[s0, s1, run, acf, A, bx]	
20	55	True	"""	498	[rotr, sig0, sig1, Sig0, Sig1, Ch, Maj, sha_co...	
21	57	True	"""	371	[rotr, sig0, sig1, Sig0, Sig1, Ch, Maj, zone, ...	
22	58	True	"""	311	[s0, s1, compress_block, A_signed, acf, bx]	
23	59	True	"""	319	[rotr, sig0, sig1, Sig0, Sig1, Ch, Maj, zone, ...	
24	66	False	# The split (exact, all 64 rounds):	25	[]	inva character (U+00E (<unknown li
25	68	False	# SESSION SYNTHESIS: SHA-256 Harmonic Structure	42	[]	unterminat string lite (detected line
26	71	False	# THE 6-CYCLE DISCOVERY	57	[]	unterminat string lite (detected line
27	73	True	#!/usr/bin/env python3	289	[rotr, sig0, sig1, Sig0, Sig1, ch, maj, add32,...	
28	78	True	def recover_cycle_from_hash(final_hash):	40	[recover_cycle_from_hash]	
29	79	True	import numpy as np	49	[bbp_basin, compute_scar, recursive_orbit]	

	block_id	ok	title	lines	defs	err
30	80	True	import numpy as np	50	[sha_kernel_decomp]	
31	81	True	import numpy as np	30	[decompose]	
32	82	True	"""	763	[add32, sub32, rotr, shr, Sig0, Sig1, sig0, si...	
33	83	True	"""	336	[add32, sub32, rotr, shr, Sig0, Sig1, sig0, si...	
34	84	True	"""	381	[add32, sub32, rotr, rotl, shr, Sig0, Sig1, si...	
35	89	True	"""	508	[add32, sub32, rotr, shr, Sig0, Sig1, sig0, si...	
36	90	True	"""	508	[add32, sub32, rotr, shr, Sig0, Sig1, sig0, si...	
37	91	True	"""	78	[add32, sub32, rotr, Sig0, Sig1, Ch, Maj, grav...	
38	92	True	"""	345	[rotr, S0, S1, s0, s1, Ch, Maj, add, expand, s...	
39	93	True	"""	252	[add, sub, rotr, Sig0, Sig1, sig0, sig1, Ch, M...	
40	94	True	"""	257	[add, sub, rotr, Sig0, Sig1, sig0, sig1, Ch, M...	
41	95	True	# ==========...	156	[signed32, build_terminal_trinity_system, solv...	
42	96	False	import numpy as np	164	[]	invalid synt (<unknown line 1(
43	97	True	"""	79	[genlock_read]	
44	98	True	import struct	41	[sha256_full, residual_with_target]	

	block_id	ok	title	lines	defs	err
45	99	True		0	[]	

2) SHA-256 core that is actually usable

This is the clean spine. It replaces the duplicated one-off scripts with one trace engine that supports:

- standard constants
- inverse constants
- per-round state capture
- digest scar inspection
- reverse / byte / word / bit variants

In [49]:
```python
def rotr(x: int, n: int) -> int:
    return ((x >> n) | (x << (32 - n))) & M32

def ch(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (~x & z)) & M32

def maj(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (x & z) ^ (y & z)) & M32

def big_sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def big_sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def small_sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def small_sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def words_to_bytes(words: Iterable[int]) -> bytes:
    return b''.join(struct.pack('>I', w & M32) for w in words)
```

```
def bytes_to_words(buf: bytes) -> List[int]:
    assert len(buf) % 4 == 0
    return list(struct.unpack(f'>{len(buf)//4}I', buf))

def pad_sha256(msg: bytes) -> bytes:
    out = bytearray(msg)
    bit_len = len(msg) * 8
    out.append(0x80)
    while len(out) % 64 != 56:
        out.append(0)
    out += struct.pack('>Q', bit_len)
    return bytes(out)

@dataclass
class TraceResult:
    digest_hex: str
    final_state: Tuple[int, ...]
    round_states: List[Tuple[int, ...]]
    T1: List[int]
    T2: List[int]
    block_count: int
    first_block_words: List[int]


def sha256_trace(msg: bytes, H_init: List[int] | None = None, K: List[int] | None = None) -> TraceResult:
    H_init = list(STD_H if H_init is None else H_init)
    K = list(STD_K if K is None else K)
    state = H_init[:]
    padded = pad_sha256(msg)
    all_states, all_T1, all_T2 = [], [], []
    first_block_words = []

    for off in range(0, len(padded), 64):
        block = padded[off:off+64]
        W = list(struct.unpack('>16I', block))
        if off == 0:
            first_block_words = W[:]
        for t in range(16, 64):
            W.append((small_sigma1(W[t-2]) + W[t-7] + small_sigma0(W[t-15]) + W[t-16]) & M32)
        a, b, c, d, e, f, g, h = state
        for t in range(64):
```

```
            T1 = (h + big_sigma1(e) + ch(e, f, g) + K[t] + W[t]) & M32
            T2 = (big_sigma0(a) + maj(a, b, c)) & M32
            h, g, f, e, d, c, b, a = g, f, e, (d + T1) & M32, c, b, a, (T1 + T2) & M32
            all_states.append((a, b, c, d, e, f, g, h))
            all_T1.append(T1)
            all_T2.append(T2)
        state = [((x + y) & M32) for x, y in zip(state, [a, b, c, d, e, f, g, h])]

    return TraceResult(
        digest_hex=''.join(f'{x:08x}' for x in state),
        final_state=tuple(state),
        round_states=all_states,
        T1=all_T1,
        T2=all_T2,
        block_count=len(padded) // 64,
        first_block_words=first_block_words,
    )


def reverse_hex_variants(hex_digest: str) -> Dict[str, str]:
    b = [hex_digest[i:i+2] for i in range(0, len(hex_digest), 2)]
    w = [hex_digest[i:i+8] for i in range(0, len(hex_digest), 8)]
    def reverse_bits_in_byte(hx: str) -> str:
        v = int(hx, 16)
        return f"{int(f'{v:08b}'[::-1], 2):02x}"
    return {
        'chars_rev': hex_digest[::-1],
        'bytes_rev': ''.join(reversed(b)),
        'words32_rev': ''.join(reversed(w)),
        'bits_each_byte_rev': ''.join(reverse_bits_in_byte(x) for x in b),
    }

# smoke check
msg = b'NEXUS|TEST|' * 12
tr = sha256_trace(msg)
assert tr.digest_hex == hashlib.sha256(msg).hexdigest()
print(tr.digest_hex, 'blocks=', tr.block_count)
```

```
3ee61390166701ed753bb5e32a4c18469c798796367414b0d169f912d2bebf6e blocks= 3
```

In [50]:

```
def digest_words(hex_digest: str) -> List[int]:
    return [int(hex_digest[i:i+8], 16) for i in range(0, len(hex_digest), 8)]
```

```
def signed32(x: int) -> int:
    x &= M32
    return x if x < 0x80000000 else x - 0x100000000

def word_A_values(hex_digest: str, H: float = H) -> List[dict]:
    out = []
    for i, w in enumerate(digest_words(hex_digest)):
        C = w / M32
        if C >= H:
            A = math.sqrt(max(C*C - H*H, 0.0))
            sign = +1
        else:
            A = math.sqrt(max(H*H - C*C, 0.0))
            sign = -1
        bits = f'{w:032b}'
        out.append({
            'word': i,
            'hex': f'0x{w:08x}',
            'C': C,
            'A_signed': sign * A,
            'ones': bits.count('1'),
            'runs_1': len([x for x in bits.split('0') if x]),
        })
    return out

pd.DataFrame(word_A_values(tr.digest_hex))
```

Out[50]:

	word	hex	C	A_signed	ones	runs_1
0	0	0x3ee61390	0.245698	-0.247950	15	6
1	1	0x166701ed	0.087509	-0.337919	15	7
2	2	0x753bb5e3	0.457942	0.296419	20	9
3	3	0x2a4c1846	0.165224	-0.307487	11	8
4	4	0x9c798796	0.611229	0.501751	17	7
5	5	0x367414b0	0.212709	-0.276771	13	8
6	6	0xd169f912	0.818023	0.739807	16	9
7	7	0xd2bebf6e	0.823223	0.745553	22	9

3) Hex-native input families

These are the families worth testing first because they stay in the machine's own alphabet.

In [51]:
```
def build_inputs() -> Dict[str, bytes]:
    K_bytes = words_to_bytes(STD_K)
    H_bytes = words_to_bytes(STD_H)
    K_inv_bytes = words_to_bytes(INV_K)
    H_inv_bytes = words_to_bytes(INV_H)

    # phase-shifted / echoed family from the recent experiments
    K_phase_shifted = K_bytes + K_bytes[4:] + K_bytes[:4]

    interleave_HK = bytearray()
    for h, k in zip(STD_H * 8, STD_K[:64]):
        interleave_HK += struct.pack('>I', h)
        interleave_HK += struct.pack('>I', k)

    reversed_K_words = words_to_bytes(list(reversed(STD_K)))
    pi_hex_bytes = bytes.fromhex(PI_HEX_64)

    return {
```

```
        'K_constants_full': K_bytes,
        'K_constants_half': words_to_bytes(STD_K[:32]),
        'H_constants': H_bytes,
        'K_INV_as_msg': K_inv_bytes,
        'H_INV_as_msg': H_inv_bytes,
        'K_phase_shifted': K_phase_shifted,
        'K_reversed_words': reversed_K_words,
        'H_then_K_interleave': bytes(interleave_HK),
        'PI_hex_slice': pi_hex_bytes,
    }

INPUTS = build_inputs()
{k: len(v) for k, v in INPUTS.items()}
```

Out[51]:
```
{'K_constants_full': 256,
 'K_constants_half': 128,
 'H_constants': 32,
 'K_INV_as_msg': 256,
 'H_INV_as_msg': 32,
 'K_phase_shifted': 512,
 'K_reversed_words': 256,
 'H_then_K_interleave': 512,
 'PI_hex_slice': 32}
```

4) Metrics that survive execution

These are the metrics I kept because they are explicit and reproducible:

- digest hamming distance
- transition energy (sum of round-to-round state bit changes)
- mirrored-state hamming
- zone balance (words above / below H)

In [52]:
```
def hamming_hex(h1: str, h2: str) -> int:
    return (int(h1, 16) ^ int(h2, 16)).bit_count()

def state_transition_energy(states: List[Tuple[int, ...]]) -> int:
    if len(states) < 2:
        return 0
```

```
    total = 0
    for prev, cur in zip(states[:-1], states[1:]):
        total += sum((a ^ b).bit_count() for a, b in zip(prev, cur))
    return total

def mirrored_state_hamming(states_a: List[Tuple[int, ...]], states_b: List[Tuple[int, ...]]) -> dict:
    n = min(len(states_a), len(states_b))
    comp = []
    rev_b = list(reversed(states_b[:n]))
    for sa, sb in zip(states_a[:n], rev_b):
        comp.append(sum((a ^ b).bit_count() for a, b in zip(sa, sb)))
    return {
        'rounds_compared': n,
        'min': min(comp) if comp else None,
        'max': max(comp) if comp else None,
        'avg': round(sum(comp)/len(comp), 2) if comp else None,
    }

def zone_balance(hex_digest: str, H: float = H) -> Tuple[int, int]:
    vals = [w / M32 for w in digest_words(hex_digest)]
    real = sum(v >= H for v in vals)
    imag = len(vals) - real
    return real, imag

def summarize_pair(msg: bytes, H_init_a=STD_H, K_a=STD_K, H_init_b=INV_H, K_b=INV_K) -> dict:
    std = sha256_trace(msg, H_init_a, K_a)
    inv = sha256_trace(msg, H_init_b, K_b)
    e_std = state_transition_energy(std.round_states)
    e_inv = state_transition_energy(inv.round_states)
    return {
        'size_bytes': len(msg),
        'blocks': std.block_count,
        'std_digest': std.digest_hex,
        'inv_digest': inv.digest_hex,
        'hamming_bits': hamming_hex(std.digest_hex, inv.digest_hex),
        'std_energy': e_std,
        'inv_energy': e_inv,
        'energy_ratio_inv_over_std': (e_inv / e_std) if e_std else float('nan'),
        'std_zone': zone_balance(std.digest_hex),
        'inv_zone': zone_balance(inv.digest_hex),
        'mirrored_alignment': mirrored_state_hamming(std.round_states, inv.round_states),
    }
```

```
summarize_pair(INPUTS['K_constants_full'])
```

Out[52]:

```
{'size_bytes': 256,
 'blocks': 5,
 'std_digest': 'e4c62ae41929873b99fa3f871694451c9dbd0300448aba7ea4ff6d68a6b52d95',
 'inv_digest': '5f80e54690df0f0cfb86a07823db4ff973eec9c4031d6fab0055120f03c8e6a9',
 'hamming_bits': 146,
 'std_energy': 40723,
 'inv_energy': 40640,
 'energy_ratio_inv_over_std': 0.9979618397465806,
 'std_zone': (5, 3),
 'inv_zone': (4, 4),
 'mirrored_alignment': {'rounds_compared': 320,
  'min': 109,
  'max': 151,
  'avg': 128.25}}
```


In [53]:
```
rows = []
for name, msg in INPUTS.items():
    rec = summarize_pair(msg)
    rec['name'] = name
    rec['mirrored_avg'] = rec['mirrored_alignment']['avg']
    rows.append(rec)

runs = pd.DataFrame(rows)[[
    'name', 'size_bytes', 'blocks', 'hamming_bits', 'std_energy', 'inv_energy',
    'energy_ratio_inv_over_std', 'std_zone', 'inv_zone', 'mirrored_avg'
]].sort_values(['hamming_bits', 'mirrored_avg'])
runs
```

Out[53]:

	name	size_bytes	blocks	hamming_bits	std_energy	inv_energy	energy_ratio_inv_over_std	std_zone	inv_zone
4	H_INV_as_msg	32	1	121	8304	8132	0.979287	(4, 4)	(4, 4)
3	K_INV_as_msg	256	5	124	41147	40953	0.995285	(6, 2)	(6, 2)
5	K_phase_shifted	512	9	125	73418	73409	0.999877	(5, 3)	(5, 3)
8	PI_hex_slice	32	1	125	8076	7926	0.981426	(4, 4)	(7, 1)
2	H_constants	32	1	126	8129	8218	1.010948	(6, 2)	(5, 3)
7	H_then_K_interleave	512	9	132	73653	73364	0.996076	(4, 4)	(6, 2)
6	K_reversed_words	256	5	133	40884	40966	1.002006	(5, 3)	(5, 3)
1	K_constants_half	128	3	139	24154	24509	1.014697	(6, 2)	(4, 4)
0	K_constants_full	256	5	146	40723	40640	0.997962	(5, 3)	(4, 4)

5) BBP finite-state orbit tooling

This keeps the useful part of the BBP material: the moment the state is restricted to hex digits, repeated application becomes a finite directed graph with basins and cycles. That is real and testable.

In [54]:

```
def bbp_step(n: int) -> int:
    return BBP_TRANS[n % 16]

def orbit(seed: int, limit: int = 64) -> List[int]:
    out = [seed]
    seen = {seed: 0}
    cur = seed
    for _ in range(limit - 1):
        cur = bbp_step(cur)
        out.append(cur)
        if cur in seen:
            break
        seen[cur] = len(out) - 1
    return out
```

```python
def orbit_signature(seed: int) -> dict:
    path = orbit(seed)
    last = path[-1]
    first_idx = path.index(last)
    cycle = path[first_idx:-1] if path.count(last) > 1 else [last]
    return {
        'seed': seed,
        'path': path,
        'tail_len': first_idx,
        'cycle': cycle,
        'cycle_len': len(cycle),
    }

orbit_table = pd.DataFrame([orbit_signature(s) for s in range(16)])
orbit_table[['seed', 'tail_len', 'cycle', 'cycle_len']]
```

Out[54]:

	seed	tail_len	cycle	cycle_len
0	0	0	[0, 2, 8, 7, 3, 15]	6
1	1	0	[1, 4, 14]	3
2	2	0	[2, 8, 7, 3, 15, 0]	6
3	3	0	[3, 15, 0, 2, 8, 7]	6
4	4	0	[4, 14, 1]	3
5	5	0	[5, 12, 11]	3
6	6	0	[6, 9, 13]	3
7	7	0	[7, 3, 15, 0, 2, 8]	6
8	8	0	[8, 7, 3, 15, 0, 2]	6
9	9	0	[9, 13, 6]	3
10	10	0	[10]	1
11	11	0	[11, 5, 12]	3
12	12	0	[12, 11, 5]	3
13	13	0	[13, 6, 9]	3
14	14	0	[14, 1, 4]	3
15	15	0	[15, 0, 2, 8, 7, 3]	6

6) New synthesis: resonance scanner

This is the extra piece I added from the whole pile.

It sweeps **input family × transform family × engine family** and ranks candidates by:

- lower digest separation
- lower energy drift from 1.0

- lower mirrored-state hamming

This gives you one place to look for the next hook instead of manually comparing scattered scripts.

In [55]:
```python
def message_transforms(msg: bytes) -> Dict[str, bytes]:
    # byte-safe transforms only
    words = bytes_to_words(msg[:len(msg) - (len(msg) % 4)]) if len(msg) >= 4 else []
    out = {
        'identity': msg,
        'bytes_rev': msg[::-1],
        'xor_ff': bytes(b ^ 0xFF for b in msg),
    }
    if words:
        out['words32_rev'] = words_to_bytes(list(reversed(words)))
    return out


def H_conjugate_constants(constants: List[int], H_value: float = H) -> List[int]:
    # reflect normalized constants across H on the unit interval
    conj = []
    for w in constants:
        c = w / M32
        rc = (2 * H_value - c) % 1.0
        conj.append(int(rc * M32) & M32)
    return conj

CONJ_K = H_conjugate_constants(STD_K)
CONJ_H = H_conjugate_constants(STD_H)

ENGINE_FAMILIES = {
    'std_vs_inv': (STD_H, STD_K, INV_H, INV_K),
    'std_vs_conj': (STD_H, STD_K, CONJ_H, CONJ_K),
    'std_vs_std': (STD_H, STD_K, STD_H, STD_K),
}

scanner_rows = []
for input_name, raw in INPUTS.items():
    for transform_name, transformed in message_transforms(raw).items():
        for engine_name, (Ha, Ka, Hb, Kb) in ENGINE_FAMILIES.items():
            rec = summarize_pair(transformed, Ha, Ka, Hb, Kb)
            scanner_rows.append({
```

```
                'input': input_name,
                'transform': transform_name,
                'engine': engine_name,
                'bytes': len(transformed),
                'hamming': rec['hamming_bits'],
                'energy_ratio': rec['energy_ratio_inv_over_std'],
                'energy_drift': abs(rec['energy_ratio_inv_over_std'] - 1.0),
                'mirrored_avg': rec['mirrored_alignment']['avg'],
            })

scan = pd.DataFrame(scanner_rows)
scan['score'] = (256 - scan['hamming']) + (1 - scan['energy_drift']) * 50 + (160 - scan['mirrored_avg'])
scan.sort_values('score', ascending=False).head(20)
```

Out[55]:

	input	transform	engine	bytes	hamming	energy_ratio	energy_drift	mirrored_avg	score
26	H_constants	identity	std_vs_std	32	0	1.0	0.0	124.97	341.03
44	K_INV_as_msg	xor_ff	std_vs_std	256	0	1.0	0.0	126.91	339.09
98	PI_hex_slice	identity	std_vs_std	32	0	1.0	0.0	126.94	339.06
8	K_constants_full	xor_ff	std_vs_std	256	0	1.0	0.0	127.12	338.88
89	H_then_K_interleave	bytes_rev	std_vs_std	512	0	1.0	0.0	127.26	338.74
50	H_INV_as_msg	identity	std_vs_std	32	0	1.0	0.0	127.28	338.72
104	PI_hex_slice	xor_ff	std_vs_std	32	0	1.0	0.0	127.38	338.62
20	K_constants_half	xor_ff	std_vs_std	128	0	1.0	0.0	127.58	338.42
5	K_constants_full	bytes_rev	std_vs_std	256	0	1.0	0.0	127.61	338.39
47	K_INV_as_msg	words32_rev	std_vs_std	256	0	1.0	0.0	127.64	338.36
32	H_constants	xor_ff	std_vs_std	32	0	1.0	0.0	127.72	338.28
80	K_reversed_words	xor_ff	std_vs_std	256	0	1.0	0.0	127.78	338.22
65	K_phase_shifted	bytes_rev	std_vs_std	512	0	1.0	0.0	127.85	338.15
2	K_constants_full	identity	std_vs_std	256	0	1.0	0.0	127.88	338.12
83	K_reversed_words	words32_rev	std_vs_std	256	0	1.0	0.0	127.88	338.12
14	K_constants_half	identity	std_vs_std	128	0	1.0	0.0	127.89	338.11
56	H_INV_as_msg	xor_ff	std_vs_std	32	0	1.0	0.0	128.03	337.97
74	K_reversed_words	identity	std_vs_std	256	0	1.0	0.0	128.14	337.86
11	K_constants_full	words32_rev	std_vs_std	256	0	1.0	0.0	128.14	337.86
77	K_reversed_words	bytes_rev	std_vs_std	256	0	1.0	0.0	128.16	337.84

7) Digest scar reader

This is the compact reader that survived from the stronger analysis sections:

- normalized digest words
- A-values from the (A^2 + H^2 = C^2) carve
- bit-density and run structure

That exact carve shows up repeatedly in the file's harmonic SHA reflection / Glass Key material.

In [56]:
```
sample_name = scan.sort_values('score', ascending=False).iloc[0]['input']
sample_msg = INPUTS[sample_name]
sample_trace = sha256_trace(sample_msg)
scar_df = pd.DataFrame(word_A_values(sample_trace.digest_hex))
print('sample input =', sample_name)
print('digest =', sample_trace.digest_hex)
scar_df
```

```
sample input = H_constants
digest = 5cce668898a2e2d7dedbb552165a2a77596d092b03df3fcf68118d63982b6106
```

Out[56]:

	word	hex	C	A_signed	ones	runs_1
0	0	0x5cce6688	0.362524	0.097862	15	8
1	1	0x98a2e2d7	0.596235	0.483373	16	10
2	2	0xdedbb552	0.870540	0.797492	20	11
3	3	0x165a2a77	0.087313	-0.337969	16	10
4	4	0x596d092b	0.349320	0.013323	15	11
5	5	0x03df3fcf	0.015125	-0.348738	21	4
6	6	0x68118d63	0.406518	0.208350	13	8
7	7	0x982b6106	0.594412	0.481122	12	8

8) Export helpers

Use these to save the current curation state.

In [57]:
```python
def export_curated_state(out_dir: str = 'curated_exports'):
    out = Path(out_dir)
    out.mkdir(exist_ok=True)
    manifest.to_csv(out / 'code_block_manifest.csv', index=False)
    runs.to_csv(out / 'harmonic_input_runs.csv', index=False)
    scan.sort_values('score', ascending=False).to_csv(out / 'resonance_scan.csv', index=False)
    scar_df.to_csv(out / 'sample_digest_scar.csv', index=False)
    with open(out / 'summary.json', 'w') as f:
        json.dump({
            'source': 'embedded_manifest',
            'kept_python_blocks': int(manifest['ok'].sum()),
            'total_python_blocks': int(len(manifest)),
            'top_resonance_row': scan.sort_values('score', ascending=False).iloc[0].to_dict(),
        }, f, indent=2, default=str)
    return out

export_curated_state()
```

Out[57]:
```
WindowsPath('curated_exports')
```

Standalone reverse primitives (no forward trace reference)

This section adds an **exact backward peel** for SHA-256 compression rounds.

What it does:

- starts from the **digest only**
- converts the digest back to the final working state for a single block by subtracting the chaining state
- walks **backward round-by-round** using only:
  - the current round state
  - the round constant `K[t]`
  - the schedule word `W[t]`

What it does **not** use:

- no saved forward trace
- no previously recorded `T1/T2`

- no forward-state cache

This is the clean algebraic reverse for a known schedule.

In [58]:
```python
from dataclasses import dataclass
from typing import Iterable, List, Tuple, Optional

@dataclass
class ReverseResult:
    digest_hex: str
    final_working_state: Tuple[int, ...]
    recovered_start_state: Tuple[int, ...]
    states_backward: List[Tuple[int, ...]]
    verified: bool
    block_words_head: List[int]

def subtract_feedforward(digest_hex: str, H_in: List[int] | Tuple[int, ...] = STD_H) -> Tuple[int, ...]:
    dw = digest_words(digest_hex)
    if len(dw) != 8:
        raise ValueError("digest must decode to 8 words")
    return tuple(((d - h) & M32) for d, h in zip(dw, H_in))

def expand_schedule_from_block(block64: bytes) -> List[int]:
    if len(block64) != 64:
        raise ValueError("need exactly one 64-byte padded block")
    W = list(struct.unpack('>16I', block64))
    for t in range(16, 64):
        W.append((small_sigma1(W[t-2]) + W[t-7] + small_sigma0(W[t-15]) + W[t-16]) & M32)
    return W

def reverse_round_no_trace(after_state: Tuple[int, ...], Wt: int, Kt: int) -> Tuple[int, ...]:
    a1, b1, c1, d1, e1, f1, g1, h1 = after_state

    a0 = b1
    b0 = c1
    c0 = d1
    e0 = f1
    f0 = g1
    g0 = h1

    T2 = (big_sigma0(a0) + maj(a0, b0, c0)) & M32
```

```
    T1 = (a1 - T2) & M32
    d0 = (e1 - T1) & M32
    h0 = (T1 - big_sigma1(e0) - ch(e0, f0, g0) - Kt - Wt) & M32

    return (a0, b0, c0, d0, e0, f0, g0, h0)

def reverse_block_no_trace(
    digest_hex: str,
    *,
    schedule_words: Optional[List[int]] = None,
    padded_block64: Optional[bytes] = None,
    H_in: List[int] | Tuple[int, ...] = STD_H,
    K: List[int] | Tuple[int, ...] = STD_K,
) -> ReverseResult:
    if schedule_words is None:
        if padded_block64 is None:
            raise ValueError("provide either schedule_words or padded_block64")
        schedule_words = expand_schedule_from_block(padded_block64)
    if len(schedule_words) != 64:
        raise ValueError("schedule_words must have length 64")
    if len(H_in) != 8 or len(K) != 64:
        raise ValueError("expected 8-word chaining state and 64 round constants")

    cur = subtract_feedforward(digest_hex, H_in)
    states_backward = [cur]
    for t in range(63, -1, -1):
        cur = reverse_round_no_trace(cur, schedule_words[t], K[t])
        states_backward.append(cur)

    recovered_start_state = cur
    return ReverseResult(
        digest_hex=digest_hex,
        final_working_state=states_backward[0],
        recovered_start_state=recovered_start_state,
        states_backward=states_backward,
        verified=tuple(H_in) == recovered_start_state,
        block_words_head=schedule_words[:16],
    )

def reverse_block_from_message_no_trace(
    msg: bytes,
    *,
```

```
    H_in: List[int] | Tuple[int, ...] = STD_H,
    K: List[int] | Tuple[int, ...] = STD_K,
) -> ReverseResult:
    padded = pad_sha256(msg)
    if len(padded) != 64:
        raise ValueError("this exact helper is single-block only; pass a 64-byte padded block or a schedule")
    digest_hex = sha256_trace(msg, list(H_in), list(K)).digest_hex
    return reverse_block_no_trace(digest_hex, padded_block64=padded, H_in=H_in, K=K)
```

In [59]:

```
# single-block harmonic demo: exact reverse without using saved forward trace
DEMO_ONEBLOCK = (b"NEXUS|HOOK|SCAR|RETURN|PHASE|ECHO|SHIFT|GLASS|KEY|H=PI/9"[:55])

demo_trace = sha256_trace(DEMO_ONEBLOCK)
demo_rev = reverse_block_no_trace(
    demo_trace.digest_hex,
    padded_block64=pad_sha256(DEMO_ONEBLOCK),
    H_in=STD_H,
    K=STD_K,
)

print("demo bytes =", len(DEMO_ONEBLOCK))
print("digest     =", demo_trace.digest_hex)
print("verified   =", demo_rev.verified)
print("recovered start state == IV =", demo_rev.recovered_start_state == tuple(STD_H))
print("final working state         =", [hex(x) for x in demo_rev.final_working_state])
```

```
demo bytes = 55
digest     = 01dfa391badd78914b59a61a1b982a1d9ddd874a8a175279dc83dd325c93b148
verified   = True
recovered start state == IV = True
final working state         = ['0x97d5bd2a', '0xff75ca0c', '0xeeab2a8', '0x764834e3', '0x4ccf34cb', '0xef11e9ed', '0x
bd000387', '0xb2e42f']
```